

load packages

```
library(AICcmodavg)
library(lme4)
```

```
## Loading required package: Matrix

##
## Attaching package: 'lme4'

## The following object is masked from 'package:AICcmodavg':
##
##      checkConv
```

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.6
## v forcats    1.0.1      v stringr    1.6.0
## v ggplot2    4.0.1      v tibble     3.3.1
## v lubridate  1.9.4      v tidyr      1.3.2
## v purrr      1.2.1
```

```
## -- Conflicts ----- tidyverse_conflicts() --
## x tidyr::expand() masks Matrix::expand()
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## x tidyr::pack()    masks Matrix::pack()
## x tidyr::unpack() masks Matrix::unpack()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(dplyr)
library(glue)
library(zoo)
```

```
##
## Attaching package: 'zoo'
##
## The following objects are masked from 'package:base':
##
##      as.Date, as.Date.numeric
```

```
library(MASS)
```

```
##
## Attaching package: 'MASS'
##
## The following object is masked from 'package:dplyr':
##
##      select
```

```
library(foreach)
```

```
## Warning: package 'foreach' was built under R version 4.5.3
```

```
##  
## Attaching package: 'foreach'  
##  
## The following objects are masked from 'package:purrr':  
##  
##    accumulate, when
```

```
library(doParallel)
```

```
## Warning: package 'doParallel' was built under R version 4.5.3
```

```
## Loading required package: iterators  
## Loading required package: parallel
```

```
library(MuMin)
```

```
##  
## Attaching package: 'MuMin'  
##  
## The following objects are masked from 'package:AICcmodavg':  
##  
##    AICc, DIC, importance
```

load necessary data

Here I'm loading a list of best-fit viral rate models as well as a master long-formatted demographic dataset containing all years and sites (produced by scripts in a previous project).

```
load("model_assessment/data/model_export_mar_2026.RData")    ## this environment contains vital rate fu
```

1. global IPM

First, we want to get population growth rate aggregating data from all sites, which assume the assumptions of SFTS (space for time substitution).

First, using the models we just loaded, we'll first build an fire (TSF, TBF) and climate driven, deterministic IPM predicting long term population growth rate (λ) against FRI (fire return interval) scenarios. This IPM will have 100 bins. Upper and lower bounds of the IPM are the upper and lower sizes observed across all years. Seedlings that enter the kernel all have the same distribution. In the code chunk below, I first set up the parameters of the IPM as mentioned above, then create a dataframe representing hypothetical FRI scenarios with or without a history of fire suppression; all other covariates were held constant at the means. We'll resample model parameter estimates ($n=1000$) to account for parameter uncertainty and ease the calculate of CI around λ .

1.1 initiate IPM

```
#size bounds for IPM
TBF_long$logsize0 <- (log(TBF_long$size0+0.1))
TBF_long$s.logsize0 <- scale(TBF_long$logsize0)
TBF_long$s.TBF <- scale(TBF_long$TBF)
TBF_long$s.TSF <- scale(TBF_long$TSF)
TBF_long$sq.TSF <- (TBF_long$TSF)^2
TBF_long$s.sq.TSF <- scale(TBF_long$sq.TSF)
IPM_bounds <- range(TBF_long$logsize0, na.rm = TRUE) # our IPM will be based on logsize as a size metric
no_bins <- 100
bin_edges <- seq(IPM_bounds[1], IPM_bounds[2], length.out= no_bins+1)
bin_mids <- zoo::rollmean(bin_edges,2)

log.fr <- log(1.1)
# define mean and sd of seedling sizes
rows_with_new <- TBF_long[grepl("\\bnew", TBF_long$comm1), ]
rows_with_new <- rows_with_new[!grepl("no new", rows_with_new$comm1), ] # delete "no new"
rows_with_new <- rows_with_new[!grepl("no news", rows_with_new$comm1), ] # delete "no new"
rows_with_new <- rows_with_new[!grepl("not possible to see new plants", rows_with_new$comm1), ]
rows_with_new <- rows_with_new[!grepl("No news", rows_with_new$comm1), ]
rows_with_new <- rows_with_new[!grepl("new lvs", rows_with_new$comm1), ]
rows_with_new <- rows_with_new[!grepl("new nail", rows_with_new$comm1), ]

mean_seedling_logsize <- mean(rows_with_new$logsize1, na.rm=T) # get the mean logsize of seedlings ###
var_seedling_logsize <- var(rows_with_new$logsize1, na.rm=T) # get the var in logsize of seedlings

### build empty matrices to store IPM output. Here, I'm interested in the effect of fire on lambda, so
possible_scenarios <-
  as.data.frame(matrix(NA, nrow= 22, ncol = 28))
names(possible_scenarios) <- c("FRI", "s.TBF", "s.T_LA", "s.T_LH",
  "s.sq.T_LH",
  "s.sq.P_LA",
  "s.T_LC",
  "s.T_LD",
  "s.P_LD",
  "s.sq.T_LA",
  "s.P_LA",
  "s.P_LW",
  "s.P_LH",
  "s.T_RA",
  "s.sq.T_RA",
  "s.P_RA",
  "s.T_RH",
  "s.P_RH",
  "s.T_RC",
  "s.T_RW",
  "s.T_RD",
  "s.sq.T_RC",
  "s.P_RW",
  "s.P_RD",
  "s.sq.T_RH",
  "s.sq.P_RA",
```

```

"s.sq.P_RW",
"s.sq.P_RD")

possible_scenarios$FRI <- rep(1:11, 2) # we want FRI to range from 1-11
possible_scenarios$s.TBF <- scale(rep(c(0, 10), each= 11) ,# raw TBF values
                                center= attr(TBF_long$s.TBF,"scaled:center"),
                                scale= attr(TBF_long$s.TBF,"scaled:scale")) # for each set of FRI's,

possible_scenarios[, c( "s.T_LA", "s.T_LH", "s.sq.T_LH", "s.sq.P_LA", "s.T_LC", "s.T_LD", "s.P_L",
                        "s.P_LW", "s.P_LH", "s.T_RA", "s.sq.T_RA", "s.P_RA", "s.T_RH", "s.P_RH",
                        "s.sq.T_RC","s.P_RW","s.P_RD", "s.sq.T_RH", "s.sq.P_RA", "s.sq.P_RW", "s.sq

my_sites <- unique(TBF_long$site)
no_reps <- 1000 # or somethign similar

##### extract model components for later predict() #####
sur_model_terms <- terms(survival_mod)
sur_term <- attr(sur_model_terms, "term.labels")
sur_term <- reformulate(sur_term)

gr_model_terms <- terms(growth_mod)
gr_term <- attr(gr_model_terms, "term.labels")
gr_term <- reformulate(gr_term)

prep_model_terms <- terms(prepare_mod)
prep_term <- attr(prepare_model_terms, "term.labels")
prep_term <- reformulate(prep_term)

crep_model_terms <- terms(crep_mod)
crep_term <- attr(crep_model_terms, "term.labels")
crep_term <- reformulate(crep_term)

vargrowth_model_terms <- terms(vargrowth_mod)
vargrowth_term <- attr(vargrowth_model_terms, "term.labels")
vargrowth_term <- reformulate(vargrowth_term)

rec_model_terms <- terms(recruit_mod)
rec_term <- attr(rec_model_terms, "term.labels")
rec_term <- reformulate(rec_term)

##### initiate IPM #####
all_pop_growth_rates <- matrix(data= NA, nrow= dim(possible_scenarios)[1], ncol= no_reps)
n_cores <- detectCores()
cl <- makeCluster(n_cores - 1)
registerDoParallel(cl)
j_pop_gr_rate <- rep(NA, dim(possible_scenarios)[1]) ### changed to no_scenarios

all_pop_growth_rates <- foreach(
  j = 1:no_reps,
  .combine = cbind,

```

```

.packages = c("MASS", "lme4", "popbio", "dplyr")
) %dopar% {
  rsur_coefs <- coefficients(survival_mod) # first, assign all "randomly selected" coefficients to the m
  rsur_coefs[!names(coefficients(survival_mod)) %in% paste("site", my_sites, sep="")] <- MASS::mvrnorm(
  #rsur_coefs <- lme4::fixef(survival_mod)
  #rsur_coefs <- MASS::mvrnorm(mu= lme4::fixef(survival_mod) , Sigma = vcov(survival_mod))
  rgr_coefs <- lme4::fixef(growth_mod)
  rgr_coefs <- MASS::mvrnorm(mu= lme4::fixef(growth_mod) , Sigma = vcov(growth_mod)) # no site effect
  rprep_coefs <- coefficients(prepare_mod)
  rprep_coefs[!names(coefficients(prepare_mod)) %in% paste("site", my_sites, sep="")] <- MASS::mvrnorm(mu=
  rcrep_coefs <- lme4::fixef(crep_mod)
  rcrep_coefs <- MASS::mvrnorm(mu= lme4::fixef(crep_mod) , Sigma = vcov(crep_mod)) # no site effect in
  rvar_coefs <- lme4::fixef(vargrowth_mod)
  rvar_coefs <- MASS::mvrnorm(mu= lme4::fixef(vargrowth_mod) , Sigma = vcov(vargrowth_mod)) # no site
  rrec_coefs <- lme4::fixef(recruit_mod)
  rrec_coefs <- MASS::mvrnorm(mu= lme4::fixef(recruit_mod) , Sigma = vcov(recruit_mod)) # no site effe
  #rrec_coefs <- MASS::mvrnorm(mu= lme4::fixef(recruit_mod) , Sigma = vcov(recruit_mod))

  for (i in 1: dim(possible_scenarios)[1]) {

    i_scenario <- possible_scenarios[i,] # which combination of FRI, TBF, and climate do I want to use

    TSFs <- 0:(i_scenario$FRI-1) # TSF range from 0 to (FRI-1) such that an FRI of 1 will always have T
    i_scenario <- i_scenario[rep(seq_len(nrow(i_scenario)), length(TSFs)), ]
    i_scenario$TSF <- TSFs
    i_scenario$s.TSF <- scale(TSFs, center= attr(TBF_long$s.TSF,"scaled:center"),
                             scale= attr(TBF_long$s.TSF,"scaled:scale"))
    i_scenario$s.sq.TSF <- scale(TSFs^2, center= attr(TBF_long$s.sq.TSF,"scaled:center"),
                                scale= attr(TBF_long$s.sq.TSF,"scaled:scale"))

    mx_prod <- diag(no_bins+1) # setting up an identity matrix before you enter the loop

    for (ii in TSFs){
      i_ii_scenario <- i_scenario[which(i_scenario$TSF==ii),]
      data_for_prediction <- i_ii_scenario[rep(seq_len(nrow(i_ii_scenario)), length(bin_mids)), ]
      data_for_prediction$s.logsize0 <- scale(bin_mids, center= attr(TBF_long$s.logsize0,"scaled:center"),
                                             scale= attr(TBF_long$s.logsize0,"scaled:scale"))
      data_for_prediction_rec <- cbind(i_ii_scenario)

      predicted_sur <- predicted_gr <- predicted_pr <- predicted_cr <- predicted_vg <- matrix(data=NA, n
      predicted_rec <- rep(NA, length(my_sites))

      ### make empty mxes before entering the site loop

      predicted_sur_allsites <- predicted_gr_allsites <- predicted_pr_allsites <- predicted_cr_allsites <- pr
      predicted_rec_allsites <- matrix(NA, nrow = length(my_sites), ncol = 1, dimnames = list(my_sites,

      for (s in my_sites){ #looping through the sites
        s_data_for_prediction <- data_for_prediction
        s_data_for_prediction_rec <- data_for_prediction_rec
        s_data_for_prediction$site <- s
        s_data_for_prediction_rec$site <- s

```

```

s_data_for_prediction$site      <- factor(s_data_for_prediction$site, levels = my_sites) ## i
s_data_for_prediction_rec$site  <- factor(s_data_for_prediction_rec$site, levels = my_sites)

s_data_for_prediction <- s_data_for_prediction %>% mutate(across(c("s.logsize0", "s.sq.TSF", "s
# Zheng-- you will have to change the predict() call in the below to be a custom predict functi

### build model matrices and use mx multiplication to obtain predicted values. Don't forget to

m_sur <- model.matrix(sur_term ,data=s_data_for_prediction)
p2_sur <- rsur_coefs %*% t(m_sur) # check on whether this coef(survival_mod is a column or a row
p2_sur <- plogis(p2_sur)      ## transform to binomial
predicted_sur_allsites[s,] <- p2_sur

m_gr <- model.matrix(gr_term, data=s_data_for_prediction)
p2_gr <- rgr_coefs %*% t(m_gr) # check on whether this coef(survival_mod is a column or a row!!
predicted_gr_allsites[s,] <- p2_gr

m_prep <- model.matrix(prepare_term,data=s_data_for_prediction)
p2_prep <- rprep_coefs %*% t(m_prep) # check on whether this coef(survival_mod is a column or a
p2_prep <- plogis(p2_prep)      ## transform to binomial
predicted_pr_allsites[s,] <- p2_prep

m_crep <- model.matrix(crep_term ,data=s_data_for_prediction)
p2_crep <- rcrep_coefs %*% t(m_crep) # check on whether this coef(survival_mod is a column or a
predicted_cr_allsites[s,] <- p2_crep

m_vg <- model.matrix(vargrowth_term,data=s_data_for_prediction)
p2_vg <- rvar_coefs %*% t(m_vg) #
predicted_vg_allsites[s,] <- exp(p2_vg)

m_rec <- model.matrix(rec_term , data=s_data_for_prediction_rec)
p2_rec <- rrec_coefs %*% t(m_rec) # check on whether this coef(survival_mod is a column or a row
predicted_rec_allsites[s,1] <- p2_rec*log(1.1)          ##### account for composite rec variab

# for mixed models, the above functions will predict site-specific results (i.e., will incorpor
rm(s_data_for_prediction) # this part removes the site-specific dataframe used in this loop, so
}

predicted_sur <- apply(predicted_sur_allsites, MARGIN= 2, FUN= mean) # average size-specific vita
predicted_gr <- apply(predicted_gr_allsites, MARGIN= 2, FUN= mean)
predicted_pr <- apply(predicted_pr_allsites, MARGIN= 2, FUN= mean)
predicted_cr <- apply(predicted_cr_allsites, MARGIN= 2, FUN= mean)
predicted_vg <- apply(predicted_vg_allsites, MARGIN= 2, FUN= mean)
predicted_vg[which(predicted_vg<0)] <- 0                ##### to prevent negative variance
predicted_rec <- mean(predicted_rec_allsites)

gmx <- matrix(NA, length(bin_mids),length(bin_mids)) ## create empty gmx before entering the no_b

# make the TSF-specific matrix
for (ss in 1:no_bins) {
  growcdf <- pnorm(bin_edges,predicted_gr[ss],sqrt(predicted_vg[ss])) # Yes. Vargrowth=(pred - ob
  grows <- growcdf[2:length(bin_edges)]-growcdf[1:(length(bin_edges)-1)]
  if(sum(grows)>0){grows <- grows/sum(grows)}
}

```

```

    gmx[,ss] <- grows
  } else if (sum(grows)==0 & grow_vals[ss]< min(bin_edges)) {
    gmx[,ss] <- c(1, rep(0, length(binmids)-1))
  } else if (sum(grows)==0 & grow_vals[ss]> max(bin_edges)) {
    gmx[,ss] <- c(rep(0, length(binmids)-1), 1) } else { # deals with complete eviction-- OUT OF
      gmx[,ss] <- NA
    }
  # this if statement breaks the code (puts NA's into the matrix) if the sum of the PDF is zero (
} # end ss loop

# make the surv*growth mx
survgmx <- gmx*t(matrix( rep(predicted_sur,no_bins),no_bins)) # Yes, this is doing element-wise m
repro <- predicted_sur*predicted_pr*predicted_cr # prob that you survive, reproduce, then number

mx <- matrix(0, no_bins+1, no_bins+1)
mx[2:(no_bins+1), 2:(no_bins+1)] = survgmx
mx[1,2:(no_bins+1)] = repro # FRUIT ROW

sdlng_cdf <- pnorm(bin_edges,mean_seedling_logsize,sqrt(var_seedling_logsize))
sdlng_pdf <- sdlng_cdf[2:length(bin_edges)]-sdlng_cdf[1:(length(bin_edges)-1)]
if(sum(sdlng_pdf)>0){
  sdlng_pdf <- sdlng_pdf/sum(sdlng_pdf)} else {
  sdlng_pdf <- NA}

mx[2:(no_bins+1),1] = predicted_rec *sdlng_pdf

mx_prod <- mx_prod %*% mx # in the first iteration of the loop, mx just becomes mx_prod
} # closes ii (TSFs) loop

j_pop_gr_rate[i] <- (popbio::lambda(mx_prod))^(1/length(TSFs))
# multiply them together to get a matrix, etc.
}
j_pop_gr_rate
}

```

1.2 graph lambda ~ FRI, colored by TBF

This is the relationship observed using aggregated data (Space for time substitution)

```

#####
##### global IPM results, rep=1000
#####

all_pop_growth_rates <- cbind(possible_scenarios, all_pop_growth_rates)
names(all_pop_growth_rates) <- c(names(possible_scenarios), paste("rep", 1:no_reps, sep = ""))

rep_cols <- grep("^rep", names(all_pop_growth_rates), value = TRUE)

mean_lambda_across_scenarios <- all_pop_growth_rates %>%
  rowwise() %>%
  mutate(
    mean_lambda = mean(c_across(all_of(rep_cols)), na.rm = TRUE),

```

```

    lower_bound = quantile(c_across(all_of(rep_cols)), 0.025, na.rm = TRUE),
    upper_bound = quantile(c_across(all_of(rep_cols)), 0.975, na.rm = TRUE)
  ) %>%
  dplyr::select(FRI,s.TBF,mean_lambda,lower_bound, upper_bound) %>%
  mutate(TBF=NA) %>%
  ungroup()

mean_lambda_across_scenarios$TBF[1:11] <- "Short TBF"
mean_lambda_across_scenarios$TBF[12:22] <- "Long TBF"

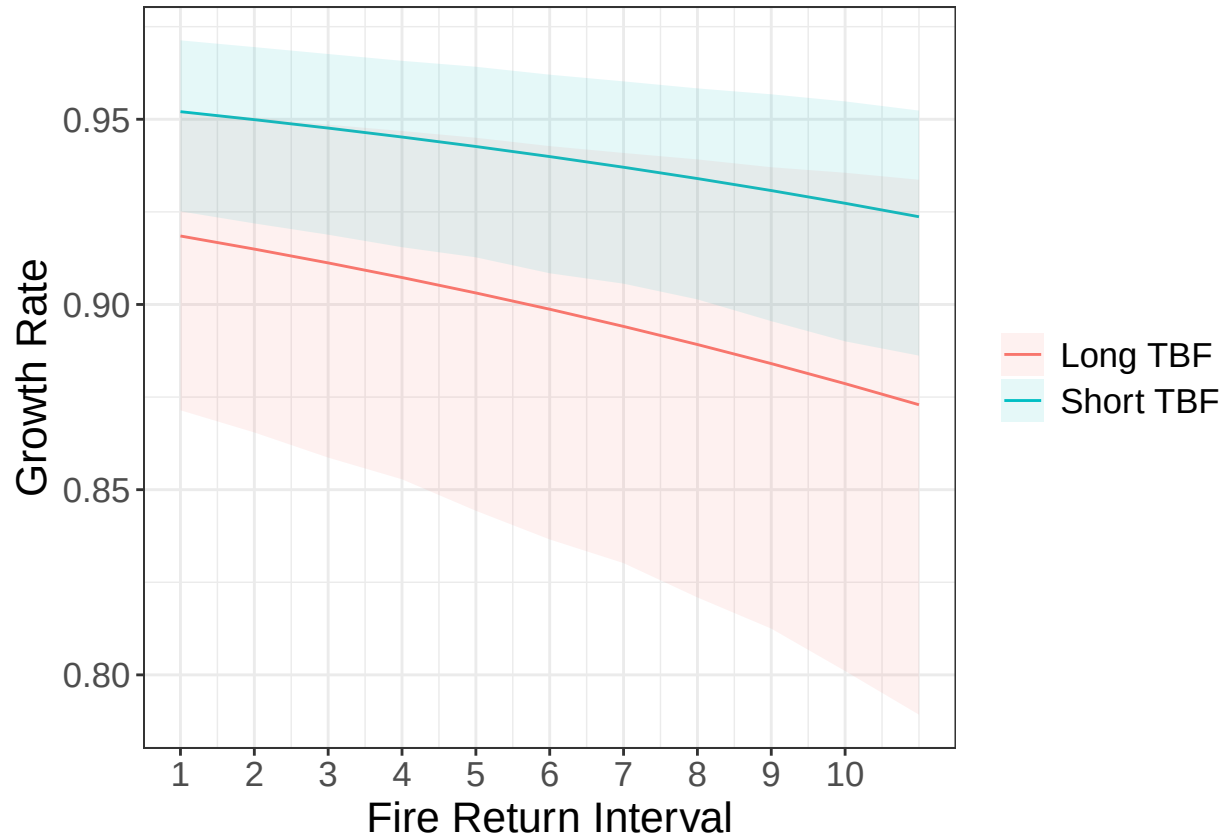
#####. graph
#####

plt1 <- ggplot(data= mean_lambda_across_scenarios, aes(x= FRI, y= mean_lambda)) +
  geom_line(aes(color= TBF)) +
  geom_ribbon(aes(ymin = mean_lambda_across_scenarios$lower_bound, ymax = mean_lambda_across_scenarios$upper_bound, fill = TBF)) +
  #ylim(0,1) +
  #xlim(c(0, 12)) +
  labs(y= "Growth Rate", x = "Fire Return Interval",color = NULL, fill = NULL) +
  theme_bw() + theme(legend.position='none') +
  theme(legend.position = "right") +
  scale_x_continuous(breaks = 0:10) +
  theme(text = element_text(size = 16))
plt1

## Warning: Use of 'mean_lambda_across_scenarios$lower_bound' is discouraged.
## i Use 'lower_bound' instead.

## Warning: Use of 'mean_lambda_across_scenarios$upper_bound' is discouraged.
## i Use 'upper_bound' instead.

```



1.3 Use the global IPM to predict region level responses

we can use the global IPM to make region specific inference, though variations among site are driven exclusively by fixed site intercepts.

1.3.1 Croatan The Croatan region contains CH and CM. Both sites had history of fire suppression

```
all_pop_growth_rates_Cro <- matrix(data= NA, nrow= dim(possible_scenarios)[1], ncol= no_reps)
n_cores <- detectCores()
cl <- makeCluster(n_cores - 1)
registerDoParallel(cl)
j_pop_gr_rate_Cro <- rep(NA, dim(possible_scenarios)[1]) ### changed to no_scenarios

all_pop_growth_rates_Cro <- foreach(
  j = 1:no_reps,
  .combine = cbind,
  .packages = c("MASS", "lme4", "popbio", "dplyr")
) %dopar% {
  rsur_coefs <- coefficients(survival_mod) # first, assign all "randomly selected" coefficients to the m
  rsur_coefs[!names(coefficients(survival_mod)) %in% paste("site", my_sites, sep="")] <- MASS::mvrnorm(
    #rsur_coefs <- lme4::fixef(survival_mod)
    #rsur_coefs <- MASS::mvrnorm(mu= lme4::fixef(survival_mod) , Sigma = vcov(survival_mod))
  rgr_coefs <- lme4::fixef(growth_mod)
  rgr_coefs <- MASS::mvrnorm(mu= lme4::fixef(growth_mod) , Sigma = vcov(growth_mod)) # no site effect
```

```

rprep_coefs <- coefficients(prepare_mod)
rprep_coefs[!names(coefficients(prepare_mod)) %in% paste("site", my_sites, sep="")] <- MASS::mvrnorm(mu=
rcrep_coefs <- lme4::fixef(crep_mod)
rcrep_coefs <- MASS::mvrnorm(mu= lme4::fixef(crep_mod) , Sigma = vcov(crep_mod)) # no site effect in
rvar_coefs <- lme4::fixef(vargrowth_mod)
rvar_coefs <- MASS::mvrnorm(mu= lme4::fixef(vargrowth_mod) , Sigma = vcov(vargrowth_mod)) # no site
rrec_coefs <- lme4::fixef(recruit_mod)
rrec_coefs <- MASS::mvrnorm(mu= lme4::fixef(recruit_mod) , Sigma = vcov(recruit_mod)) # no site effect
#rrec_coefs <- MASS::mvrnorm(mu= lme4::fixef(recruit_mod) , Sigma = vcov(recruit_mod))

for (i in 1: dim(possible_scenarios)[1]) {

  i_scenario <- possible_scenarios[i,] # which combination of FRI, TBF, and climate do I want to use

  TSFs <- 0:(i_scenario$FRI-1) # TSF range from 0 to (FRI-1) such that an FRI of 1 will always have T
  i_scenario <- i_scenario[rep(seq_len(nrow(i_scenario)), length(TSFs)), ]
  i_scenario$TSF <- TSFs
  i_scenario$s.TSF <- scale(TSFs, center= attr(TBF_long$s.TSF,"scaled:center"),
                           scale= attr(TBF_long$s.TSF,"scaled:scale"))
  i_scenario$s.sq.TSF <- scale(TSFs^2, center= attr(TBF_long$s.sq.TSF,"scaled:center"),
                              scale= attr(TBF_long$s.sq.TSF,"scaled:scale"))

  mx_prod <- diag(no_bins+1) # setting up an identity matrix before you enter the loop

  for (ii in TSFs){
    i_ii_scenario <- i_scenario[which(i_scenario$TSF==ii),]
    data_for_prediction <- i_ii_scenario[rep(seq_len(nrow(i_ii_scenario)), length(bin_mids)), ]
    data_for_prediction$s.logsize0 <- scale(bin_mids, center= attr(TBF_long$s.logsize0,"scaled:center"),
                                           scale= attr(TBF_long$s.logsize0,"scaled:scale"))
    data_for_prediction_rec <- cbind(i_ii_scenario)

    predicted_sur <- predicted_gr <- predicted_pr <- predicted_cr <- predicted_vg <- matrix(data=NA, n
    predicted_rec <- rep(NA, length(my_sites))

    ### make empty mxes before entering the site loop

    predicted_sur_allsites <- predicted_gr_allsites <- predicted_pr_allsites <- predicted_cr_allsites <- pr
    predicted_rec_allsites <- matrix(NA, nrow = length(my_sites), ncol = 1, dimnames = list(my_sites,

  for (s in my_sites[3:4]){ #looping through the sites
    s_data_for_prediction <- data_for_prediction
    s_data_for_prediction_rec <- data_for_prediction_rec
    s_data_for_prediction$site <- s
    s_data_for_prediction_rec$site <- s

    s_data_for_prediction$site <- factor(s_data_for_prediction$site, levels = my_sites) ## i
    s_data_for_prediction_rec$site <- factor(s_data_for_prediction_rec$site, levels = my_sites)

    s_data_for_prediction <- s_data_for_prediction %>% mutate(across(c("s.logsize0", "s.sq.TSF", "s
    # Zheng-- you will have to change the predict() call in the below to be a custom predict functi

    ### build model matrices and use mx multiplication to obtain predicted values. Don't forget to

```

```

m_sur <- model.matrix(sur_term ,data=s_data_for_prediction)
p2_sur <- rsur_coefs %*% t(m_sur) # check on whether this coef(survival_mod is a column or a row)
p2_sur <- plogis(p2_sur) ## transform to binomial
predicted_sur_allsites[s,] <- p2_sur

m_gr <- model.matrix(gr_term, data=s_data_for_prediction)
p2_gr <- rgr_coefs %*% t(m_gr) # check on whether this coef(survival_mod is a column or a row!)
predicted_gr_allsites[s,] <- p2_gr

m_prep <- model.matrix(prepare_term,data=s_data_for_prediction)
p2_prep <- rprep_coefs %*% t(m_prep) # check on whether this coef(survival_mod is a column or a row)
p2_prep <- plogis(p2_prep) ## transform to binomial
predicted_pr_allsites[s,] <- p2_prep

m_crep <- model.matrix(crep_term ,data=s_data_for_prediction)
p2_crep <- rcrep_coefs %*% t(m_crep) # check on whether this coef(survival_mod is a column or a row)
predicted_cr_allsites[s,] <- p2_crep

m_vg <- model.matrix(vargrowth_term,data=s_data_for_prediction)
p2_vg <- rvar_coefs %*% t(m_vg) #
predicted_vg_allsites[s,] <- exp(p2_vg)

m_rec <- model.matrix(rec_term , data=s_data_for_prediction_rec)
p2_rec <- rrec_coefs %*% t(m_rec) # check on whether this coef(survival_mod is a column or a row)
predicted_rec_allsites[s,1] <- p2_rec*log(1.1) ##### account for composite rec variable

# for mixed models, the above functions will predict site-specific results (i.e., will incorporate
rm(s_data_for_prediction) # this part removes the site-specific dataframe used in this loop, so
}

predicted_sur <- apply(predicted_sur_allsites, MARGIN= 2, FUN= mean,na.rm = TRUE) # average size-
predicted_gr <- apply(predicted_gr_allsites, MARGIN= 2, FUN= mean,na.rm = TRUE)
predicted_pr <- apply(predicted_pr_allsites, MARGIN= 2, FUN= mean,na.rm = TRUE)
predicted_cr <- apply(predicted_cr_allsites, MARGIN= 2, FUN= mean,na.rm = TRUE)
predicted_vg <- apply(predicted_vg_allsites, MARGIN= 2, FUN= mean,na.rm = TRUE)
predicted_vg[which(predicted_vg<0)] <- 0 ##### to prevent negative variance
predicted_rec <- mean(predicted_rec_allsites,na.rm = TRUE)

gmX <- matrix(NA, length(bin_mids),length(bin_mids)) ## create empty gmX before entering the no_b

# make the TSF-specific matrix
for (ss in 1:no_bins) {
  growcdf <- pnorm(bin_edges,predicted_gr[ss],sqrt(predicted_vg[ss])) # Yes. Vargrowth=(pred - ob
  grows <- growcdf[2:length(bin_edges)]-growcdf[1:(length(bin_edges)-1)]
  if(sum(grows)>0){grows <- grows/sum(grows)}
  gmX[,ss] <- grows
} else if (sum(grows)==0 & grow_vals[ss]< min(bin_edges)) {
  gmX[,ss] <- c(1, rep(0, length(binmids)-1))
} else if (sum(grows)==0 & grow_vals[ss]> max(bin_edges)) {
  gmX[,ss] <- c(rep(0, length(binmids)-1), 1) } else { # deals with complete eviction-- OUT OF
  gmX[,ss] <- NA}
# this if statement breaks the code (puts NA's into the matrix) if the sum of the PDF is zero (
} # end ss loop

```

```

# make the surv*growth mx
survgmx <- gmx*t(matrix( rep(predicted_sur,no_bins),no_bins)) # Yes, this is doing element-wise m
repro <- predicted_sur*predicted_pr*predicted_cr # prob that you survive, reproduce, then number

mx <- matrix(0, no_bins+1, no_bins+1)
mx[2:(no_bins+1), 2:(no_bins+1)] = survgmx
mx[1,2:(no_bins+1)] = repro # FRUIT ROW

sdln_g_cdf <- pnorm(bin_edges,mean_seedling_logsize,sqrt(var_seedling_logsize))
sdln_g_pdf <- sdln_g_cdf[2:length(bin_edges)]-sdln_g_cdf[1:(length(bin_edges)-1)]
if(sum(sdln_g_pdf)>0){
  sdln_g_pdf <- sdln_g_pdf/sum(sdln_g_pdf)} else {
  sdln_g_pdf <- NA}

mx[2:(no_bins+1),1] = predicted_rec *sdln_g_pdf

mx_prod <- mx_prod %*% mx # in the first iteration of the loop, mx just becomes mx_prod
} # closes ii (TSFs) loop

j_pop_gr_rate_Cro[i] <- (popbio::lambda(mx_prod))^(1/length(TSFs))
# multiply them together to get a matrix, etc.
}
j_pop_gr_rate_Cro
}

all_pop_growth_rates_Cro <- cbind(possible_scenarios, all_pop_growth_rates_Cro)
names(all_pop_growth_rates_Cro) <- c(names(possible_scenarios), paste("rep", 1:no_reps, sep = ""))
rep_cols <- grep("rep", names(all_pop_growth_rates_Cro), value = TRUE)

mean_lambda_across_scenarios_Cro <- all_pop_growth_rates_Cro %>%
  rowwise() %>%
  mutate(
    mean_lambda = mean(c_across(all_of(rep_cols)), na.rm = TRUE),
    lower_bound = quantile(c_across(all_of(rep_cols)), 0.025, na.rm = TRUE),
    upper_bound = quantile(c_across(all_of(rep_cols)), 0.975, na.rm = TRUE)
  ) %>%
  dplyr::select(FRI,s.TBF,mean_lambda,lower_bound, upper_bound) %>%
  mutate(TBF=NA) %>%
  ungroup()

mean_lambda_across_scenarios_Cro$TBF[1:11] <- "Short TBF"
mean_lambda_across_scenarios_Cro$TBF[12:22] <- "Long TBF"

#####. graph
#####

plt2 <- ggplot(data= mean_lambda_across_scenarios_Cro, aes(x= FRI, y= mean_lambda)) +
  geom_line(aes(color= TBF)) +
  geom_ribbon(aes(ymin = mean_lambda_across_scenarios_Cro$lower_bound, ymax = mean_lambda_across_scenar
  #ylim(0,1) +
  #xlim(c(0, 12)) +

```

```

labs(y= "Growth Rate", x = "Fire Return Interval",color = NULL, fill = NULL) +
theme_bw() + theme(legend.position='none') +
theme(legend.position = "right") +
scale_x_continuous(breaks = 0:10) +
theme(text = element_text(size = 16))
plt2

```

```

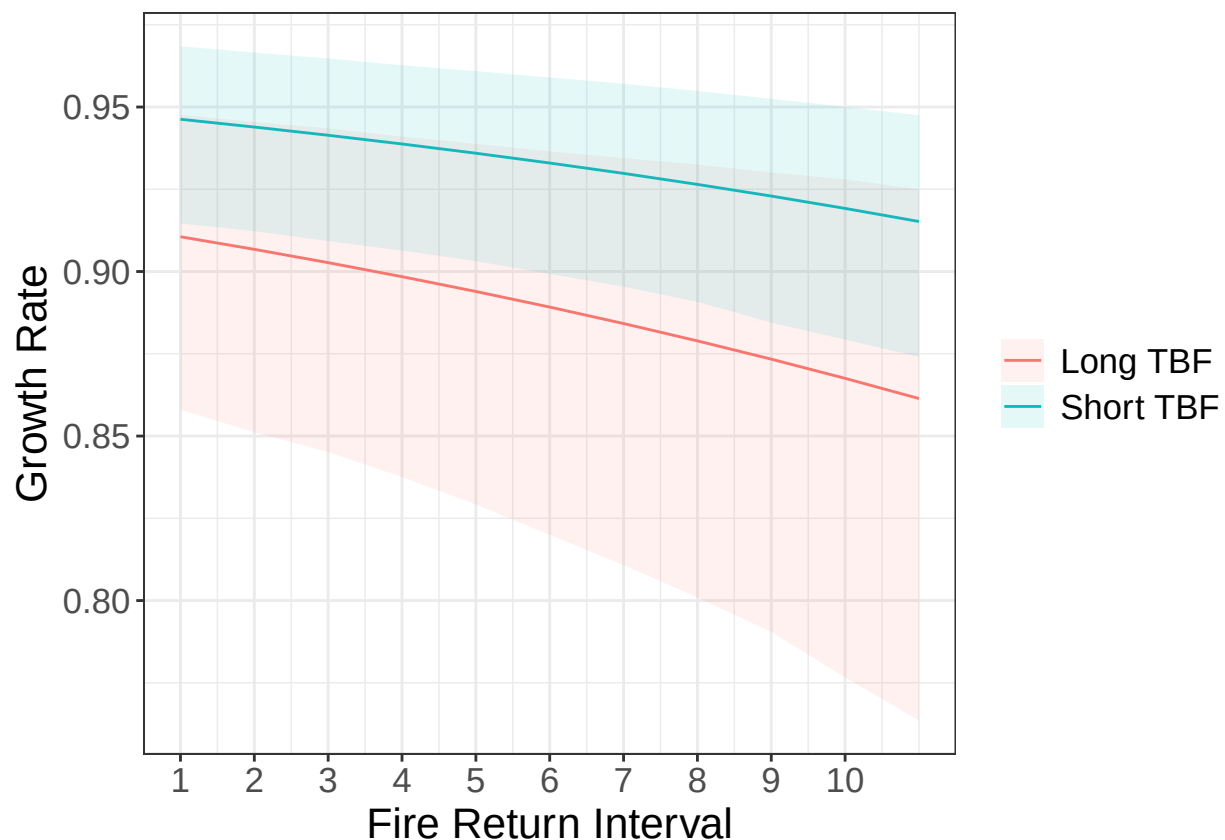
## Warning: Use of 'mean_lambda_across_scenarios_Cro$lower_bound' is discouraged.
## i Use 'lower_bound' instead.

```

```

## Warning: Use of 'mean_lambda_across_scenarios_Cro$upper_bound' is discouraged.
## i Use 'upper_bound' instead.

```



1.3.2 Green Swamps (no history of fire suppression) The GSP (Green Swamp) region contains GSP-BI and GSP-LI, both sites were subject to fire management and didnot have histories of fire suppression.

```

all_pop_growth_rates_GSP <- matrix(data= NA, nrow= dim(possible_scenarios)[1], ncol= no_reps)
n_cores <- detectCores()
cl <- makeCluster(n_cores - 1)
registerDoParallel(cl)
j_pop_gr_rate_GSP <- rep(NA, dim(possible_scenarios)[1]) ### changed to no_scenarios

all_pop_growth_rates_GSP <- foreach(

```

```

j = 1:no_reps,
.combine = cbind,
.packages = c("MASS", "lme4", "popbio", "dplyr")
) %dopar% {
  rsur_coefs <- coefficients(survival_mod) # first, assign all "randomly selected" coefficients to the m
  rsur_coefs[!names(coefficients(survival_mod)) %in% paste("site", my_sites, sep="")] <- MASS::mvrnorm(
  #rsur_coefs <- lme4::fixef(survival_mod)
  #rsur_coefs <- MASS::mvrnorm(mu= lme4::fixef(survival_mod) , Sigma = vcov(survival_mod))
  rgr_coefs <- lme4::fixef(growth_mod)
  rgr_coefs <- MASS::mvrnorm(mu= lme4::fixef(growth_mod) , Sigma = vcov(growth_mod)) # no site effect
  rprep_coefs <- coefficients(prepare_mod)
  rprep_coefs[!names(coefficients(prepare_mod)) %in% paste("site", my_sites, sep="")] <- MASS::mvrnorm(mu=
  rcrep_coefs <- lme4::fixef(crep_mod)
  rcrep_coefs <- MASS::mvrnorm(mu= lme4::fixef(crep_mod) , Sigma = vcov(crep_mod)) # no site effect in
  rvar_coefs <- lme4::fixef(vargrowth_mod)
  rvar_coefs <- MASS::mvrnorm(mu= lme4::fixef(vargrowth_mod) , Sigma = vcov(vargrowth_mod)) # no site
  rrec_coefs <- lme4::fixef(recruit_mod)
  rrec_coefs <- MASS::mvrnorm(mu= lme4::fixef(recruit_mod) , Sigma = vcov(recruit_mod)) # no site effe
  #rrec_coefs <- MASS::mvrnorm(mu= lme4::fixef(recruit_mod) , Sigma = vcov(recruit_mod))

  for (i in 1: dim(possible_scenarios)[1]) {

    i_scenario <- possible_scenarios[i,] # which combination of FRI, TBF, and climate do I want to use

    TSFs <- 0:(i_scenario$FRI-1) # TSF range from 0 to (FRI-1) such that an FRI of 1 will always have T
    i_scenario <- i_scenario[rep(seq_len(nrow(i_scenario)), length(TSFs)), ]
    i_scenario$TSF <- TSFs
    i_scenario$s.TSF <- scale(TSFs, center= attr(TBF_long$s.TSF,"scaled:center"),
                             scale= attr(TBF_long$s.TSF,"scaled:scale"))
    i_scenario$s.sq.TSF <- scale(TSFs^2, center= attr(TBF_long$s.sq.TSF,"scaled:center"),
                                scale= attr(TBF_long$s.sq.TSF,"scaled:scale"))

    mx_prod <- diag(no_bins+1) # setting up an identity matrix before you enter the loop

    for (ii in TSFs){
      i_ii_scenario <- i_scenario[which(i_scenario$TSF==ii),]
      data_for_prediction <- i_ii_scenario[rep(seq_len(nrow(i_ii_scenario)), length(bin_mids)), ]
      data_for_prediction$s.logsize0 <- scale(bin_mids, center= attr(TBF_long$s.logsize0,"scaled:center"),
                                             scale= attr(TBF_long$s.logsize0,"scaled:scale"))
      data_for_prediction_rec <- cbind(i_ii_scenario)

      predicted_sur <- predicted_gr <- predicted_pr <- predicted_cr <- predicted_vg <- matrix(data=NA, n
      predicted_rec <- rep(NA, length(my_sites))

      ### make empty mxes before entering the site loop

      predicted_sur_allsites <- predicted_gr_allsites <- predicted_pr_allsites <- predicted_cr_allsites <- pr
      predicted_rec_allsites <- matrix(NA, nrow = length(my_sites), ncol = 1, dimnames = list(my_sites,

      for (s in my_sites[5:6]){ #looping through the sites
        s_data_for_prediction <- data_for_prediction
        s_data_for_prediction_rec <- data_for_prediction_rec

```

```

s_data_for_prediction$site <- s
s_data_for_prediction_rec$site <- s

s_data_for_prediction$site <- factor(s_data_for_prediction$site, levels = my_sites) ## i
s_data_for_prediction_rec$site <- factor(s_data_for_prediction_rec$site, levels = my_sites)

s_data_for_prediction <- s_data_for_prediction %>% mutate(across(c("s.logsize0", "s.sq.TSF", "s
# Zheng-- you will have to change the predict() call in the below to be a custom predict functi

### build model matrices and use mx multiplication to obtain predicted values. Don't forget to

m_sur <- model.matrix(sur_term, data=s_data_for_prediction)
p2_sur <- rsur_coefs %*% t(m_sur) # check on whether this coef(survival_mod is a column or a row
p2_sur <- plogis(p2_sur) ## transform to binomial
predicted_sur_allsites[s,] <- p2_sur

m_gr <- model.matrix(gr_term, data=s_data_for_prediction)
p2_gr <- rgr_coefs %*% t(m_gr) # check on whether this coef(survival_mod is a column or a row!!
predicted_gr_allsites[s,] <- p2_gr

m_prep <- model.matrix(prepare_term, data=s_data_for_prediction)
p2_prep <- rprep_coefs %*% t(m_prep) # check on whether this coef(survival_mod is a column or a
p2_prep <- plogis(p2_prep) ## transform to binomial
predicted_pr_allsites[s,] <- p2_prep

m_crep <- model.matrix(crep_term, data=s_data_for_prediction)
p2_crep <- rcrep_coefs %*% t(m_crep) # check on whether this coef(survival_mod is a column or a
predicted_cr_allsites[s,] <- p2_crep

m_vg <- model.matrix(vargrowth_term, data=s_data_for_prediction)
p2_vg <- rvar_coefs %*% t(m_vg) #
predicted_vg_allsites[s,] <- exp(p2_vg)

m_rec <- model.matrix(rec_term, data=s_data_for_prediction_rec)
p2_rec <- rrec_coefs %*% t(m_rec) # check on whether this coef(survival_mod is a column or a row
predicted_rec_allsites[s,1] <- p2_rec*log(1.1) ##### account for composite rec variab

# for mixed models, the above functions will predict site-specific results (i.e., will incorpor
rm(s_data_for_prediction) # this part removes the site-specific dataframe used in this loop, so
}

predicted_sur <- apply(predicted_sur_allsites, MARGIN= 2, FUN= mean, na.rm = TRUE) # average size-
predicted_gr <- apply(predicted_gr_allsites, MARGIN= 2, FUN= mean, na.rm = TRUE)
predicted_pr <- apply(predicted_pr_allsites, MARGIN= 2, FUN= mean, na.rm = TRUE)
predicted_cr <- apply(predicted_cr_allsites, MARGIN= 2, FUN= mean, na.rm = TRUE)
predicted_vg <- apply(predicted_vg_allsites, MARGIN= 2, FUN= mean, na.rm = TRUE)
predicted_vg[which(predicted_vg<0)] <- 0 ##### to prevent negative variance
predicted_rec <- mean(predicted_rec_allsites, na.rm = TRUE)

gmx <- matrix(NA, length(bin_mids), length(bin_mids)) ## create empty gmx before entering the no_b

# make the TSF-specific matrix
for (ss in 1:no_bins) {

```

```

growcdf <- pnorm(bin_edges,predicted_gr[ss],sqrt(predicted_vg[ss])) # Yes. Vargrowth=(pred - ob
grows <- growcdf[2:length(bin_edges)]-growcdf[1:(length(bin_edges)-1)]
if(sum(grows)>0){grows <- grows/sum(grows)
gmx[,ss] <- grows
} else if (sum(grows)==0 & grow_vals[ss]< min(bin_edges)) {
  gmx[,ss] <- c(1, rep(0, length(binmids)-1))
} else if (sum(grows)==0 & grow_vals[ss]> max(bin_edges)) {
  gmx[,ss] <- c(rep(0, length(binmids)-1), 1) } else { # deals with complete eviction-- OUT OF
    gmx[,ss] <- NA}
  # this if statement breaks the code (puts NA's into the matrix) if the sum of the PDF is zero (
} # end ss loop

# make the surv*growth mx
survgmx <- gmx*t(matrix( rep(predicted_sur,no_bins),no_bins)) # Yes, this is doing element-wise m
repro <- predicted_sur*predicted_pr*predicted_cr # prob that you survive, reproduce, then number

mx <- matrix(0, no_bins+1, no_bins+1)
mx[2:(no_bins+1), 2:(no_bins+1)] = survgmx
mx[1,2:(no_bins+1)] = repro # FRUIT ROW

sdlng_cdf <- pnorm(bin_edges,mean_seedling_logsize,sqrt(var_seedling_logsize))
sdlng_pdf <- sdlng_cdf[2:length(bin_edges)]-sdlng_cdf[1:(length(bin_edges)-1)]
if(sum(sdlng_pdf)>0){
  sdlng_pdf <- sdlng_pdf/sum(sdlng_pdf)} else {
  sdlng_pdf <- NA}

mx[2:(no_bins+1),1] = predicted_rec *sdlng_pdf

mx_prod <- mx_prod %*% mx # in the first iteration of the loop, mx just becomes mx_prod
} # closes ii (TSFs) loop

j_pop_gr_rate_GSP[i] <- (popbio::lambda(mx_prod))^(1/length(TSFs))
# multiply them together to get a matrix, etc.
}
j_pop_gr_rate_GSP
}

all_pop_growth_rates_GSP <- cbind(possible_scenarios, all_pop_growth_rates_GSP)
names(all_pop_growth_rates_GSP) <- c(names(possible_scenarios), paste("rep", 1:no_reps, sep = ""))
rep_cols <- grep("^rep", names(all_pop_growth_rates_GSP), value = TRUE)

mean_lambda_across_scenarios_GSP <- all_pop_growth_rates_GSP %>%
  rowwise() %>%
  mutate(
    mean_lambda = mean(c_across(all_of(rep_cols)), na.rm = TRUE),
    lower_bound = quantile(c_across(all_of(rep_cols)), 0.025, na.rm = TRUE),
    upper_bound = quantile(c_across(all_of(rep_cols)), 0.975, na.rm = TRUE)
  ) %>%
  dplyr::select(FRI,s.TBF,mean_lambda,lower_bound, upper_bound) %>%
  mutate(TBF=NA) %>%
  ungroup()

```

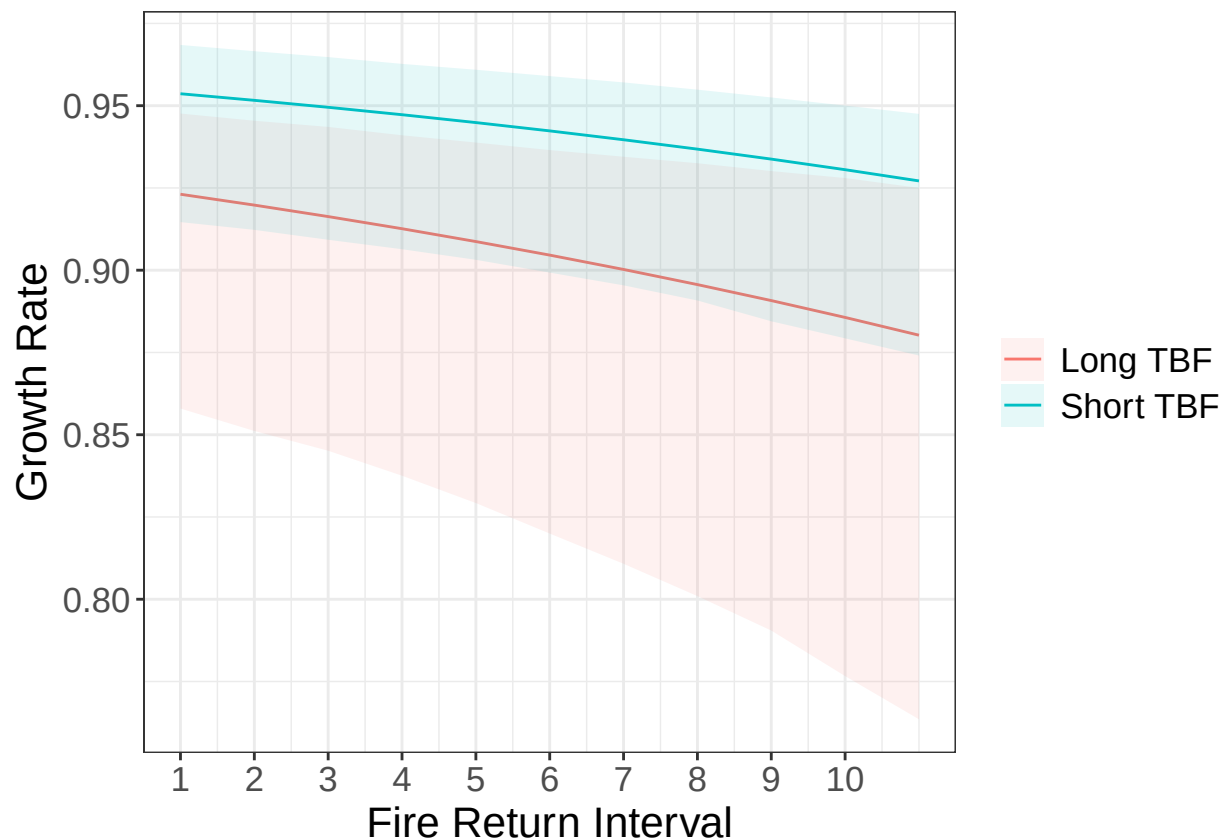
```

mean_lambda_across_scenarios_GSP$TBF[1:11] <- "Short TBF"
mean_lambda_across_scenarios_GSP$TBF[12:22] <- "Long TBF"

#####. graph
#####

plt3 <- ggplot(data= mean_lambda_across_scenarios_GSP, aes(x= FRI, y= mean_lambda)) +
  geom_line(aes(color= TBF)) +
  geom_ribbon(aes(ymin = mean_lambda_across_scenarios_Cro$lower_bound, ymax = mean_lambda_across_scenarios_Cro$upper_bound, fill = TBF)) +
  #ylim(0,1) +
  #xlim(c(0, 12)) +
  labs(y= "Growth Rate", x = "Fire Return Interval", color = NULL, fill = NULL) +
  theme_bw() + theme(legend.position='none') +
  theme(legend.position = "right") +
  scale_x_continuous(breaks = 0:10) +
  theme(text = element_text(size = 16))
plt3

```



```

lambda_globalIPM <- bind_rows(
  mean_lambda_across_scenarios %>% mutate(dataset = "global"),
  mean_lambda_across_scenarios_Cro %>% mutate(dataset = "Croatan"),
  mean_lambda_across_scenarios_GSP %>% mutate(dataset = "Green Swamp")
)

```

```

)

plt4 <- ggplot(lambda_globalIPM, aes(x = FRI, y = mean_lambda)) +

  # ribbons
  geom_ribbon(
    aes(ymin = lower_bound, ymax = upper_bound,
        fill = dataset, group = interaction(dataset, TBF)),
    alpha = 0.1) +
  geom_line(
    aes(color = dataset, linetype = TBF,
        group = interaction(dataset, TBF)),
    size = 1) +
  theme_bw() +
  labs(
    y = "Growth Rate",
    x = "Fire Return Interval",
    color = "Dataset",
    fill = "Dataset",
    linetype = "TBF") +
  scale_x_continuous(breaks = 0:10) +
  theme(text = element_text(size = 16))

```

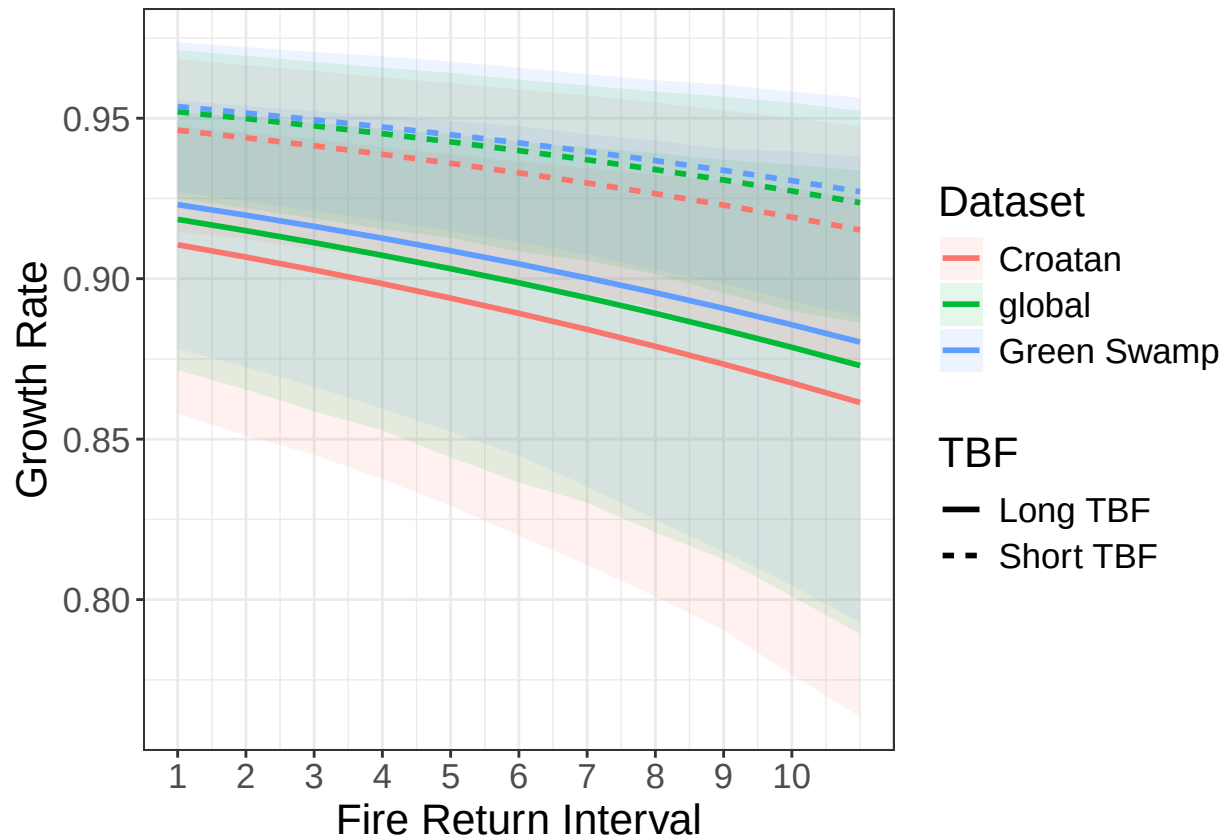
1.3.3 graph region specific predictions together with the aggregated prediction

```

## Warning: Using 'size' aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use 'linewidth' instead.
## This warning is displayed once per session.
## Call 'lifecycle::last_lifecycle_warnings()' to see where this warning was
## generated.

```

```
plt4
```



Again, the differences in growth rate among regions are driven solely by the site fixed effect in two of the vital rate functions (survival and prob of reproduction).

2. build region specific IPM (refit vital rate functions using subsets of data)

Now we want to build region specific IPMs with refitted vital rate functions using subsets of the master demographic data. This ensure that the vital rate functions and the IPM will capture region specific responses of plants to environmental drivers.

2.1 Croatan

```
TBF_long_Cro <- TBF_long %>%
  filter(site=="CH" | site=="CM")
```

2.1.1 refit vital rate functions Refit vital rate functions. Here we start with the same global model and then perform model selection using AICc to determine the best fit model for each vital rate.

survival

```
sur_subset_Cro <-
  TBF_long_Cro %>%
  filter_at(vars(sur0_1, s.logsize0, s.TSF,s.TBF, site), all_vars(!is.na(.)))
```

```
## Warning: Using one column matrices in 'filter()' was deprecated in dplyr 1.1.0.
## i Please use one dimensional logical vectors instead.
## i The deprecated feature was likely used in the dplyr package.
## Please report the issue at <https://github.com/tidyverse/dplyr/issues>.
## This warning is displayed once per session.
## Call 'lifecycle::last_lifecycle_warnings()' to see where this warning was
## generated.
```

```
GM_R_sur <- glm(
  sur0_1 ~ s.logsize0 + s.TSF * s.TBF +
  s.T_LA + s.T_LH + s.sq.T_LH + s.T_LD +s.P_RH +
  s.P_RH:s.T_LH + s.P_RD +
  site,
  data = sur_subset_Cro,
  family = "binomial",
  na.action = "na.fail"
)

n_cores <- detectCores()
n_cores
```

```
## [1] 20
```

```
cluster <- makeCluster(n_cores - 1)
registerDoParallel(cluster)
clusterExport(cluster, c("sur_subset_Cro")) ### replace with different subset (different global model)
clusterEvalQ(cluster, {library(lme4); library(MuMIn)})
```

```
## [[1]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[2]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[3]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[4]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[5]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
```

```

## [[6]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[7]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[8]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[9]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[10]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[11]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[12]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[13]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[14]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[15]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[16]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[17]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[18]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[19]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"

```

```
## [7] "utils"      "datasets"    "methods"     "base"
```

```
sur_dredge_Cro <- MuMin::dredge(
  GM_R_sur,
  cluster = cluster,
  trace   = 2
)
```

```
## Fixed term is "(Intercept)"
```

```
## |
```

```
sur_mod_Cro <- get.models(sur_dredge_Cro, 1)[[1]]
summary(sur_mod_Cro)
```

```
##
## Call:
## glm(formula = sur0_1 ~ s.logsize0 + s.P_RD + s.P_RH + s.sq.T_LH +
##       s.T_LA + s.T_LD + s.T_LH + s.TSF + site + s.P_RH:s.T_LH +
##       1, family = "binomial", data = sur_subset_Cro, na.action = "na.fail")
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)    3.08416    0.26010  11.857 < 2e-16 ***
## s.logsize0      0.75559    0.06995  10.801 < 2e-16 ***
## s.P_RD         -0.52275    0.13299  -3.931 8.47e-05 ***
## s.P_RH         -0.24776    0.12727  -1.947 0.051573 .
## s.sq.T_LH     -28.69994    5.99393  -4.788 1.68e-06 ***
## s.T_LA         1.36629    0.54075   2.527 0.011515 *
## s.T_LD        -0.74982    0.16879  -4.442 8.90e-06 ***
## s.T_LH        29.21980    6.01592   4.857 1.19e-06 ***
## s.TSF         -0.17897    0.06611  -2.707 0.006785 **
## siteCM        -2.03218    0.58872  -3.452 0.000557 ***
## s.P_RH:s.T_LH -0.96908    0.11472  -8.447 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##    Null deviance: 2393.1  on 4273  degrees of freedom
## Residual deviance: 2154.9  on 4263  degrees of freedom
## AIC: 2176.9
##
## Number of Fisher Scoring iterations: 6
```

```
stopCluster(cluster)
```

```
growth
```

```
gr_subset_Cro <-
  TBF_long_Cro[which(TBF_long_Cro$sur0_1 == 1),] %>%      ## make sure to subset data to only alive pla
```

```

    filter_at(vars(logsize1, s.logsize0, s.TSF,s.sq.TSF, s.TBF, site, s.sq.T_RA,s.T_RA, s.P_RA, s.T_RC,
                  s.T_RD, s.P_RH), all_vars(!is.na(.)))
GM_R_gr <- lmer(
  logsize1 ~ s.logsize0 + s.TSF * s.TBF + s.T_LA + s.sq.T_LA + s.P_RA + s.T_LA:s.P_RA +
  s.T_LC + s.T_LD + s.P_RH + (1|site),
  data = gr_subset_Cro,
  na.action = "na.fail"
)

n_cores <- detectCores()
n_cores

```

```
## [1] 20
```

```

cluster <- makeCluster(n_cores - 1)
registerDoParallel(cluster)
clusterExport(cluster, c("gr_subset_Cro")) ### replace with different subset (different global models)
clusterEvalQ(cluster, {library(lme4); library(MuMIn)})

```

```

## [[1]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[2]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[3]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[4]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[5]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[6]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[7]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[8]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[9]]

```

```
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[10]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[11]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[12]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[13]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[14]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[15]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[16]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[17]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[18]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[19]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
```

```
gr_dredge_Cro <- MuMIn::dredge(
  GM_R_gr,
  cluster = cluster,
  trace   = 2
)
```

```
## Warning in .dredge.par(global.model = GM_R_gr, trace = 2, cluster = cluster):
## comparing models fitted by REML
```

```
## Fixed term is "(Intercept)"
```

```
## |
```

```
growth_mod_Cro<- get.models(gr_dredge_Cro, 1)[[1]]
summary(growth_mod_Cro)
```

```
## Linear mixed model fit by REML ['lmerMod']
## Formula: logsize1 ~ s.logsize0 + s.P_RA + s.sq.T_LA + s.T_LA + s.T_LD +
##      s.TBF + (1 | site) + s.P_RA:s.T_LA
## Data: gr_subset_Cro
##
## REML criterion at convergence: 9327.7
##
## Scaled residuals:
##      Min       1Q   Median       3Q      Max
## -8.4477 -0.5630  0.0542  0.6328  3.3476
##
## Random effects:
## Groups   Name                Variance Std.Dev.
## site     (Intercept)  1.6224     1.2737
## Residual                    0.7165     0.8465
## Number of obs: 3709, groups: site, 2
##
## Fixed effects:
##              Estimate Std. Error t value
## (Intercept)   4.53626    0.90106   5.034
## s.logsize0     0.34370    0.01807  19.022
## s.P_RA        -0.09765    0.02941  -3.321
## s.sq.T_LA      8.33013    1.54523   5.391
## s.T_LA       -7.43905    1.53575  -4.844
## s.T_LD       -0.32147    0.04119  -7.805
## s.TBF        -0.05782    0.01494  -3.871
## s.P_RA:s.T_LA -0.11256    0.03766  -2.989
##
## Correlation of Fixed Effects:
##              (Intr) s.lgs0 s.P_RA s..T_L s.T_LA s.T_LD s.TBF
## s.logsize0  -0.001
## s.P_RA      0.007 -0.098
## s.sq.T_LA   0.017  0.036  0.041
## s.T_LA     -0.018 -0.033 -0.087 -0.997
## s.T_LD      0.012 -0.039  0.512 -0.233  0.160
## s.TBF      -0.001 -0.009  0.543  0.203 -0.210  0.068
## s.P_RA:.T_L -0.010  0.047 -0.459 -0.657  0.652  0.106 -0.516
```

```
stopCluster(cluster)
```

```
prep
```

```
prep_subset_Cro <-
  TBF_long_Cro %>%
  filter_at(vars(prep1, s.logsize0, s.TSF, s.TBF,s.sq.TSF, site, s.T_RA,s.P_RA, s.T_RH, s.T_RC,
    s.P_RH, ), all_vars(!is.na(.)))

GM_R_prep <- glm(
```

```

prep1 ~ s.logsize0 + s.TSF * s.TBF +
  s.T_LA + s.sq.T_LA + s.P_RA + s.sq.P_RA + s.T_LA:s.P_RA +
  s.T_LH + s.P_RH + s.T_LH:s.P_RH + s.T_LC +
  site,
data = prep_subset_Cro,
family = "binomial",
na.action = "na.fail"
)

n_cores <- detectCores()
n_cores

```

```
## [1] 20
```

```

cluster <- makeCluster(n_cores - 1)
registerDoParallel(cluster)
clusterExport(cluster, c("prep_subset_Cro")) ### replace with different subset (different global mode.
clusterEvalQ(cluster, {library(lme4); library(MuMIn)})

```

```

## [[1]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[2]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[3]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[4]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[5]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[6]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[7]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[8]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[9]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"

```

```

## [7] "utils"      "datasets"  "methods"  "base"
##
## [[10]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"    "base"
##
## [[11]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"    "base"
##
## [[12]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"    "base"
##
## [[13]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"    "base"
##
## [[14]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"    "base"
##
## [[15]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"    "base"
##
## [[16]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"    "base"
##
## [[17]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"    "base"
##
## [[18]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"    "base"
##
## [[19]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"    "base"

```

```

prep_dredge_Cro <- MuMIn::dredge(
  GM_R_prep,
  cluster = cluster,
  trace   = 2
)

```

```
## Fixed term is "(Intercept)"
```

```
## |
```

```
|
```

```
prep_mod_Cro <- get.models(prepare_dredge_Cro, 1)[[1]]

stopCluster(cluster)
```

crep

```
crep_subset_Cro <-
  TBF_long_Cro %>%
  filter_at(vars(logcrep1, s.logsize0, TSF, TBF,s.sq.TSF, site, s.T_RA, s.T_RH), all_vars(!is.na(.)))

GM_R_crep <- lmer(
  logcrep1 ~ s.logsize0 + s.TSF * s.TBF + s.T_LA + s.T_LH +
    (1 | site),
  data = crep_subset_Cro,
  na.action = "na.fail"
)

n_cores <- detectCores()
n_cores
```

```
## [1] 20
```

```
cluster <- makeCluster(n_cores - 1)
registerDoParallel(cluster)
clusterExport(cluster, c("crep_subset_Cro")) ### replace with different subset (different global mode
clusterEvalQ(cluster, {library(lme4); library(MuMIn)})
```

```
## [[1]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"     "datasets"  "methods"   "base"
##
## [[2]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"     "datasets"  "methods"   "base"
##
## [[3]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"     "datasets"  "methods"   "base"
##
## [[4]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"     "datasets"  "methods"   "base"
##
## [[5]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"     "datasets"  "methods"   "base"
##
## [[6]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"     "datasets"  "methods"   "base"
##
## [[7]]
```

```

## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[8]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[9]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[10]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[11]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[12]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[13]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[14]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[15]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[16]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[17]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[18]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[19]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"

```

```

crep_dredge_Cro <- MuMIn::dredge(
  GM_R_crep,
  cluster = cluster,

```

```

    trace    = 2
  )

## Warning in .dredge.par(global.model = GM_R_crep, trace = 2, cluster = cluster):
## comparing models fitted by REML

## Fixed term is "(Intercept)"

##      |

crep_mod_Cro <- get.models(crep_dredge_Cro, 1)[[1]]

stopCluster(cluster)

```

recruitment

since recruitment sample size is very low and subsetting would result in inadequate sample sizes, we use the recruitment model from the global IPM

```
recruit_mod_Cro <- recruit_mod
```

variance in growth

```

growth_terms <- attr(terms(growth_mod_Cro), "term.labels")
int_terms <- grep(":", growth_terms, value = TRUE)
int_terms_broken <- unique(unlist(strsplit(int_terms, ":", fixed = TRUE)))
main_terms <- growth_terms[!grepl(":", growth_terms)]
logsize1 <- c("logsize1")
gr_all_terms <- unique(c(main_terms, int_terms_broken, logsize1))

Vargrowth_subset_Cro <-
  TBF_long_Cro[which(TBF_long_Cro$sur0_1 == 1),] %>%      ## make sure to subset data to only alive pla
  filter_at(vars( s.logsize0, TSF, TBF,logsize1, site), all_vars(!is.na(.)))

Vargrowth_subset_Cro$vargrowth <- NA
Vargrowth_subset_Cro$predgrowth <- NA

Vargrowth_subset_Cro$predgrowth <- predict(growth_mod_Cro,newdata=Vargrowth_subset_Cro)

Vargrowth_subset_Cro$vargrowth <-
  (Vargrowth_subset_Cro$predgrowth - # variance is equivalent to (predicted-expected)^2
   Vargrowth_subset_Cro$logsize1)^2

Vargrowth_subset_Cro$log.vargrowth <- log(Vargrowth_subset_Cro$vargrowth)

##### to prevent negative numbers later

GM_vargrowth_R <- lmer(log.vargrowth ~ s.logsize0 + s.T_LA+ s.TSF * s.TBF +
  s.P_RA + s.sq.P_RA + s.T_LC +s.T_LD +
  s.P_RD + s.P_RW
  + (1|site), #
  data = Vargrowth_subset_Cro, na.action = "na.fail")

```

```

n_cores <- detectCores()
cluster <- makeCluster(n_cores - 1)
registerDoParallel(cluster)
clusterExport(cluster, c("Vargrowth_subset_Cro")) ### replace with different subset (different global
clusterEvalQ(cluster, {library(lme4); library(MuMIn)})

```

```

## [[1]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[2]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[3]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[4]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[5]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[6]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[7]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[8]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[9]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[10]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[11]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[12]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"

```

```
## [7] "utils"      "datasets"  "methods"  "base"
##
## [[13]]
## [1] "MuMIn"      "lme4"      "Matrix"   "stats"    "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"  "base"
##
## [[14]]
## [1] "MuMIn"      "lme4"      "Matrix"   "stats"    "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"  "base"
##
## [[15]]
## [1] "MuMIn"      "lme4"      "Matrix"   "stats"    "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"  "base"
##
## [[16]]
## [1] "MuMIn"      "lme4"      "Matrix"   "stats"    "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"  "base"
##
## [[17]]
## [1] "MuMIn"      "lme4"      "Matrix"   "stats"    "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"  "base"
##
## [[18]]
## [1] "MuMIn"      "lme4"      "Matrix"   "stats"    "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"  "base"
##
## [[19]]
## [1] "MuMIn"      "lme4"      "Matrix"   "stats"    "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"  "base"
```

```
vargrowth_dredge_Cro <- MuMIn::dredge(
  GM_vargrowth_R,
  cluster = cluster,
  trace   = 2
)
```

```
## Warning in .dredge.par(global.model = GM_vargrowth_R, trace = 2, cluster =
## cluster): comparing models fitted by REML
```

```
## Fixed term is "(Intercept)"
```

```
##      |                                                                 |
```

```
vargrowth_mod_Cro <- get.models(vargrowth_dredge_Cro, 1)[[1]]

stopCluster(cluster)
```

2.1.2 Croatan IPM keep the same IPM structure as the global IPM (for example, size bound is the same as the global IPM)

```

TBF_long$logsize0 <- (log(TBF_long$size0+0.1))
TBF_long$s.logsize0 <- scale(TBF_long$logsize0)
TBF_long$s.TBF <- scale(TBF_long$TBF)
TBF_long$s.TSF <- scale(TBF_long$TSF)
TBF_long$sq.TSF <- (TBF_long$TSF)^2
TBF_long$s.sq.TSF <- scale(TBF_long$sq.TSF)
IPM_bounds <- range(TBF_long$logsize0, na.rm = TRUE) # our IPM will be based on logsize as a size metric
no_bins <- 100
bin_edges <- seq(IPM_bounds[1], IPM_bounds[2], length.out= no_bins+1)
bin_mids <- zoo::rollmean(bin_edges,2)

# define mean and sd of seedling sizes
rows_with_new <- TBF_long[grepl("\\bnew", TBF_long$comm1), ]
rows_with_new <- rows_with_new[!grepl("no new", rows_with_new$comm1), ] # delete "no new"
rows_with_new <- rows_with_new[!grepl("no news", rows_with_new$comm1), ] # delete "no new"
rows_with_new <- rows_with_new[!grepl("not possible to see new plants", rows_with_new$comm1), ]
rows_with_new <- rows_with_new[!grepl("No news", rows_with_new$comm1), ]
rows_with_new <- rows_with_new[!grepl("new lvs", rows_with_new$comm1), ]
rows_with_new <- rows_with_new[!grepl("new nail", rows_with_new$comm1), ]

mean_seedling_logsize <- mean(rows_with_new$logsize1, na.rm=T) # get the mean logsize of seedlings ###
var_seedling_logsize <- var(rows_with_new$logsize1, na.rm=T) # get the var in logsize of seedlings

### build empty matrices to store IPM output. Here, I'm interested in the effect of fire on lambda, so
possible_scenarios <-
  as.data.frame(matrix(NA, nrow= 22, ncol = 28))
names(possible_scenarios) <- c("FRI", "s.TBF", "s.T_LA", "s.T_LH",
  "s.sq.T_LH",
  "s.sq.P_LA",
  "s.T_LC",
  "s.T_LD",
  "s.P_LD",
  "s.sq.T_LA",
  "s.P_LA",
  "s.P_LW",
  "s.P_LH",
  "s.T_RA",
  "s.sq.T_RA",
  "s.P_RA",
  "s.T_RH",
  "s.P_RH",
  "s.T_RC",
  "s.T_RW",
  "s.T_RD",
  "s.sq.T_RC",
  "s.P_RW",
  "s.P_RD",
  "s.sq.T_RH",
  "s.sq.P_RA",
  "s.sq.P_RW",
  "s.sq.P_RD")

```

```

possible_scenarios$FRI <- rep(1:11, 2) # we want FRI to range from 1-11
possible_scenarios$s.TBF <- scale(rep(c(0, 10), each= 11) ,# raw TBF values
                                center= attr(TBF_long$s.TBF,"scaled:center"),
                                scale= attr(TBF_long$s.TBF,"scaled:scale")) # for each set of FRI's,

possible_scenarios[, c( "s.T_LA", "s.T_LH", "s.sq.T_LH", "s.sq.P_LA", "s.T_LC", "s.T_LD", "s.P_L",
                        "s.P_LW", "s.P_LH", "s.T_RA", "s.sq.T_RA", "s.P_RA", "s.T_RH", "s.P_RH",
                        "s.sq.T_RC", "s.P_RW", "s.P_RD", "s.sq.T_RH", "s.sq.P_RA", "s.sq.P_RW", "s.sq.

my_sites <- unique(TBF_long_Cro$site)
no_reps <- 1000 # or somethign similar

##### extract model components for later predict() #####
sur_model_terms <- terms(sur_mod_Cro)
sur_term <- attr(sur_model_terms, "term.labels")
sur_term <- reformulate(sur_term)

gr_model_terms <- terms(growth_mod_Cro)
gr_term <- attr(gr_model_terms, "term.labels")
gr_term <- reformulate(gr_term)

prep_model_terms <- terms(prepare_mod_Cro)
prep_term <- attr(prepare_model_terms, "term.labels")
prep_term <- reformulate(prep_term)

crep_model_terms <- terms(crep_mod_Cro)
crep_term <- attr(crep_model_terms, "term.labels")
crep_term <- reformulate(crep_term)

vargrowth_model_terms <- terms(vargrowth_mod_Cro)
vargrowth_term <- attr(vargrowth_model_terms, "term.labels")
vargrowth_term <- reformulate(vargrowth_term)

rec_model_terms <- terms(recruit_mod_Cro)
rec_term <- attr(rec_model_terms, "term.labels")
rec_term <- reformulate(rec_term)

all_pop_growth_rates_Cro_reg <- matrix(data= NA, nrow= dim(possible_scenarios)[1], ncol= no_reps)
n_cores <- detectCores()
cl <- makeCluster(n_cores - 1)
registerDoParallel(cl)
j_pop_gr_rate_Cro_reg <- rep(NA, dim(possible_scenarios)[1]) ### changed to no_scenarios

all_pop_growth_rates_Cro_reg <- foreach(
  j = 1:no_reps,
  .combine = cbind,
  .packages = c("MASS", "lme4", "popbio", "dplyr")
) %dopar% {
  rsur_coefs <- coefficients(sur_mod_Cro) # first, assign all "randomly selected" coefficients to the me
  rsur_coefs[!names(coefficients(sur_mod_Cro)) %in% paste("site", my_sites, sep="")] <- MASS::mvrnorm(mu
  #rsur_coefs <- lme4::fixef(survival_mod)
  #rsur_coefs <- MASS::mvrnorm(mu= lme4::fixef(survival_mod) , Sigma = vcov(survival_mod))
  rgr_coefs <- lme4::fixef(growth_mod_Cro)

```

```

rgr_coefs <- MASS::mvrnorm(mu= lme4::fixef(growth_mod_Cro) , Sigma = vcov(growth_mod_Cro)) # no site
rprep_coefs <- coefficients(prepare_mod_Cro)
rprep_coefs[!names(coefficients(prepare_mod_Cro)) %in% paste("site", my_sites, sep="")] <- MASS::mvrnorm
rcrep_coefs <- lme4::fixef(crep_mod_Cro)
rcrep_coefs <- MASS::mvrnorm(mu= lme4::fixef(crep_mod_Cro) , Sigma = vcov(crep_mod_Cro)) # no site e
rvar_coefs <- lme4::fixef(vargrowth_mod_Cro)
rvar_coefs <- MASS::mvrnorm(mu= lme4::fixef(vargrowth_mod_Cro) , Sigma = vcov(vargrowth_mod_Cro)) #
rrec_coefs <- lme4::fixef(recruit_mod_Cro)
rrec_coefs <- MASS::mvrnorm(mu= lme4::fixef(recruit_mod_Cro) , Sigma = vcov(recruit_mod_Cro)) # no s
#rrec_coefs <- MASS::mvrnorm(mu= lme4::fixef(recruit_mod) , Sigma = vcov(recruit_mod))

for (i in 1: dim(possible_scenarios)[1]) {

  i_scenario <- possible_scenarios[i,] # which combination of FRI, TBF, and climate do I want to use

  TSFs <- 0:(i_scenario$FRI-1) # TSF range from 0 to (FRI-1) such that an FRI of 1 will always have T
  i_scenario <- i_scenario[rep(seq_len(nrow(i_scenario)), length(TSFs)), ]
  i_scenario$TSF <- TSFs
  i_scenario$s.TSF <- scale(TSFs, center= attr(TBF_long$s.TSF,"scaled:center"),
                           scale= attr(TBF_long$s.TSF,"scaled:scale"))
  i_scenario$s.sq.TSF <- scale(TSFs^2, center= attr(TBF_long$s.sq.TSF,"scaled:center"),
                              scale= attr(TBF_long$s.sq.TSF,"scaled:scale"))

  mx_prod <- diag(no_bins+1) # setting up an identity matrix before you enter the loop

  for (ii in TSFs){
    i_ii_scenario <- i_scenario[which(i_scenario$TSF==ii),]
    data_for_prediction <- i_ii_scenario[rep(seq_len(nrow(i_ii_scenario)), length(bin_mids)), ]
    data_for_prediction$s.logsize0 <- scale(bin_mids, center= attr(TBF_long$s.logsize0,"scaled:center"),
                                           scale= attr(TBF_long$s.logsize0,"scaled:scale"))
    data_for_prediction_rec <- cbind(i_ii_scenario)

    predicted_sur <- predicted_gr <- predicted_pr <- predicted_cr <- predicted_vg <- matrix(data=NA, n
    predicted_rec <- rep(NA, length(my_sites))

    ### make empty mxes before entering the site loop

    predicted_sur_allsites <- predicted_gr_allsites <- predicted_pr_allsites <- predicted_cr_allsites <- pr
    predicted_rec_allsites <- matrix(NA, nrow = length(my_sites), ncol = 1, dimnames = list(my_sites,

    for (s in my_sites){ #looping through the sites
      s_data_for_prediction <- data_for_prediction
      s_data_for_prediction_rec <- data_for_prediction_rec
      s_data_for_prediction$site <- s
      s_data_for_prediction_rec$site <- s

      s_data_for_prediction$site <- factor(s_data_for_prediction$site, levels = my_sites) ## i
      s_data_for_prediction_rec$site <- factor(s_data_for_prediction_rec$site, levels = my_sites)

      s_data_for_prediction <- s_data_for_prediction %>% mutate(across(c("s.logsize0", "s.sq.TSF", "s

      m_sur <- model.matrix(sur_term ,data=s_data_for_prediction)

```

```

p2_sur <- rsur_coefs %*% t(m_sur) # check on whether this coef(survival_mod is a column or a row
p2_sur <- plogis(p2_sur)      ## transform to binomial
predicted_sur_allsites[s,] <- p2_sur

m_gr <- model.matrix(gr_term, data=s_data_for_prediction)
p2_gr <- rgr_coefs %*% t(m_gr) # check on whether this coef(survival_mod is a column or a row!!
predicted_gr_allsites[s,] <- p2_gr

m_prep <- model.matrix(prepare_term, data=s_data_for_prediction)
p2_prep <- rprep_coefs %*% t(m_prep) # check on whether this coef(survival_mod is a column or a
p2_prep <- plogis(p2_prep)      ## transform to binomial
predicted_pr_allsites[s,] <- p2_prep

m_crep <- model.matrix(crep_term , data=s_data_for_prediction)
p2_crep <- rcrep_coefs %*% t(m_crep) # check on whether this coef(survival_mod is a column or a
predicted_cr_allsites[s,] <- p2_crep

m_vg <- model.matrix(vargrowth_term, data=s_data_for_prediction)
p2_vg <- rvar_coefs %*% t(m_vg) #
predicted_vg_allsites[s,] <- exp(p2_vg)

m_rec <- model.matrix(rec_term , data=s_data_for_prediction_rec)
p2_rec <- rrec_coefs %*% t(m_rec) # check on whether this coef(survival_mod is a column or a row
predicted_rec_allsites[s,1] <- p2_rec*log(1.1)          ##### account for composite rec variab

# for mixed models, the above functions will predict site-specific results (i.e., will incorpor
rm(s_data_for_prediction) # this part removes the site-specific dataframe used in this loop, so
}
predicted_sur <- apply(predicted_sur_allsites, MARGIN= 2, FUN= mean, na.rm=T) # average size-spec
predicted_gr <- apply(predicted_gr_allsites, MARGIN= 2, FUN= mean, na.rm=T)
predicted_pr <- apply(predicted_pr_allsites, MARGIN= 2, FUN= mean, na.rm=T)
predicted_cr <- apply(predicted_cr_allsites, MARGIN= 2, FUN= mean, na.rm=T)
predicted_vg <- apply(predicted_vg_allsites, MARGIN= 2, FUN= mean, na.rm=T)
predicted_vg[which(predicted_vg<0)] <- 0          ##### to prevent negative variance
predicted_rec <- mean(predicted_rec_allsites, na.rm=T)

gmx <- matrix(NA, length(bin_mids),length(bin_mids)) ## create empty gmx before entering the no_b

# make the TSF-specific matrix
for (ss in 1:no_bins) {
  growcdf <- pnorm(bin_edges,predicted_gr[ss],sqrt(predicted_vg[ss])) # Yes. Vargrowth=(pred - ob
  grows <- growcdf[2:length(bin_edges)]-growcdf[1:(length(bin_edges)-1)]
  if(sum(grows)>0){grows <- grows/sum(grows)}
  gmx[,ss] <- grows
} else if (sum(grows)==0 & grow_vals[ss]< min(bin_edges)) {
  gmx[,ss] <- c(1, rep(0, length(binmids)-1))
} else if (sum(grows)==0 & grow_vals[ss]> max(bin_edges)) {
  gmx[,ss] <- c(rep(0, length(binmids)-1), 1) } else { # deals with complete eviction-- OUT OF
  gmx[,ss] <- NA}
# this if statement breaks the code (puts NA's into the matrix) if the sum of the PDF is zero (
} # end ss loop

```

```

# make the surv*growth mx
survgmx <- gmx*t(matrix( rep(predicted_sur,no_bins),no_bins)) # Yes, this is doing element-wise m
repro <- predicted_sur*predicted_pr*predicted_cr # prob that you survive, reproduce, then number

mx <- matrix(0, no_bins+1, no_bins+1)
mx[2:(no_bins+1), 2:(no_bins+1)] = survgmx
mx[1,2:(no_bins+1)] = repro # FRUIT ROW

sdlng_cdf <- pnorm(bin_edges,mean_seedling_logsize,sqrt(var_seedling_logsize))
sdlng_pdf <- sdlng_cdf[2:length(bin_edges)]-sdlng_cdf[1:(length(bin_edges)-1)]
if(sum(sdlng_pdf)>0){
  sdlng_pdf <- sdlng_pdf/sum(sdlng_pdf)} else {
  sdlng_pdf <- NA}

mx[2:(no_bins+1),1] = predicted_rec *sdlng_pdf

mx_prod <- mx_prod %*% mx # in the first iteration of the loop, mx just becomes mx_prod
} # closes ii (TSFs) loop

j_pop_gr_rate_Cro_reg[i] <- (popbio::lambda(mx_prod))^(1/length(TSFs))
# multiply them together to get a matrix, etc.
}
j_pop_gr_rate_Cro_reg
}

```

```

all_pop_growth_rates_Cro_reg <- cbind(possible_scenarios, all_pop_growth_rates_Cro_reg)
names(all_pop_growth_rates_Cro_reg) <- c(names(possible_scenarios), paste("rep", 1:no_reps, sep = ""))
rep_cols <- grep("^rep", names(all_pop_growth_rates_Cro_reg), value = TRUE)

mean_lambda_across_scenarios_Cro_reg <- all_pop_growth_rates_Cro_reg %>%
  rowwise() %>%
  mutate(
    mean_lambda = mean(c_across(all_of(rep_cols)), na.rm = TRUE),
    lower_bound = quantile(c_across(all_of(rep_cols)), 0.025, na.rm = TRUE),
    upper_bound = quantile(c_across(all_of(rep_cols)), 0.975, na.rm = TRUE)
  ) %>%
  dplyr::select(FRI,s.TBF,mean_lambda,lower_bound, upper_bound) %>%
  mutate(TBF=NA) %>%
  ungroup()

mean_lambda_across_scenarios_Cro_reg$TBF[1:11] <- "Short TBF"
mean_lambda_across_scenarios_Cro_reg$TBF[12:22] <- "Long TBF"

#####. graph
#####

plt5<- ggplot(data= mean_lambda_across_scenarios_Cro_reg, aes(x= FRI, y= mean_lambda)) +
  geom_line(aes(color= TBF)) +
  geom_ribbon(aes(ymin = mean_lambda_across_scenarios_Cro_reg$lower_bound, ymax = mean_lambda_across_scenarios_Cro_reg$upper_bound, fill=TBF, color=TBF))
  #ylim(0,1) +

```

```

#xlim(c(0, 12)) +
labs(y= "Growth Rate", x = "Fire Return Interval",color = NULL, fill = NULL) +
theme_bw() + theme(legend.position='none') +
theme(legend.position = "right") +
scale_x_continuous(breaks = 0:10) +
theme(text = element_text(size = 16))
plt5

```

2.1.3 graph

```

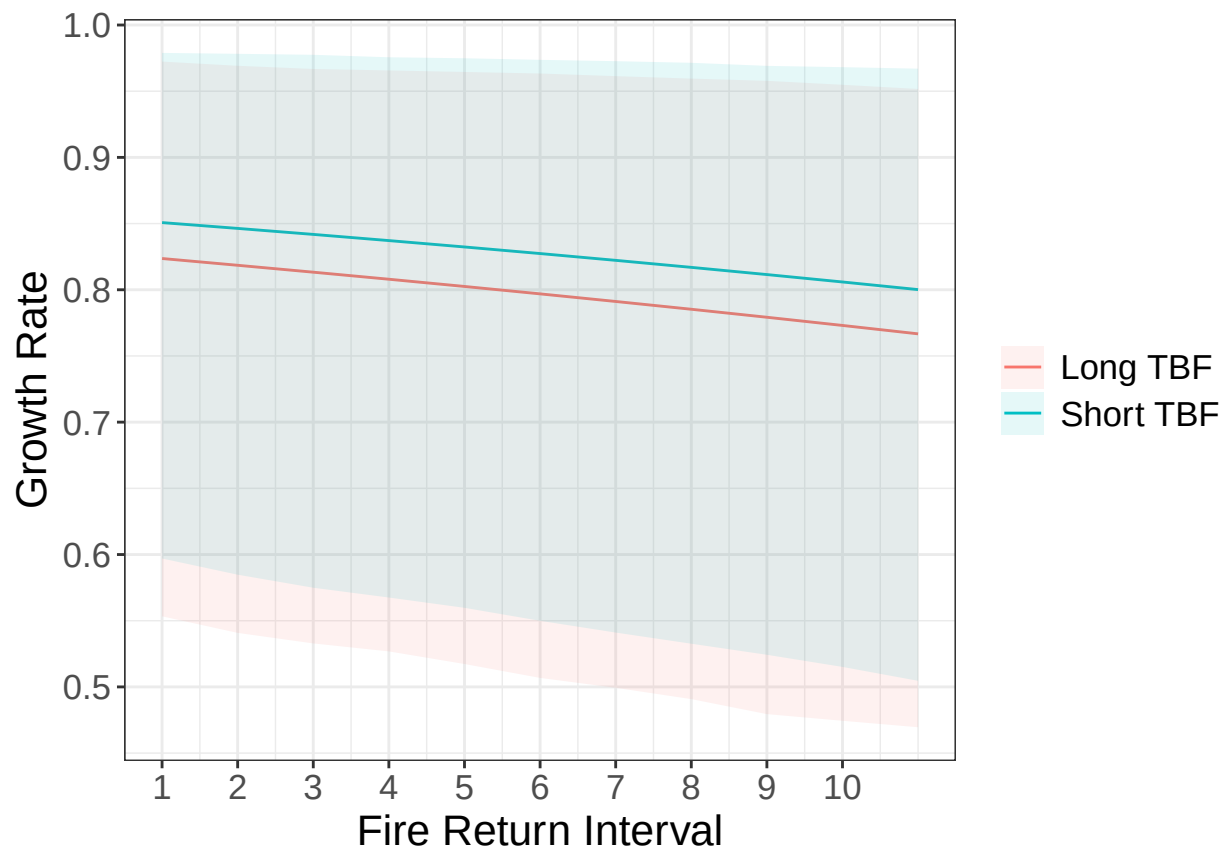
## Warning: Use of 'mean_lambda_across_scenarios_Cro_reg$lower_bound' is discouraged.
## i Use 'lower_bound' instead.

```

```

## Warning: Use of 'mean_lambda_across_scenarios_Cro_reg$upper_bound' is discouraged.
## i Use 'upper_bound' instead.

```



2.2 Green Swamp

Same procedure but for the GSP region.

```

TBF_long_GSP <- TBF_long %>%
  filter(site=="GSP-LI" | site=="GSP-BI")

```

2.2.1 refit vital rate survival

```
sur_subset_GSP <-
  TBF_long_GSP %>%
  filter_at(vars(sur0_1, s.logsize0, s.TSF,s.TBF, site), all_vars(!is.na(.)))

GM_R_sur <- glm(
  sur0_1 ~ s.logsize0 + s.TSF * s.TBF +
    s.T_LA + s.T_LH + s.sq.T_LH + s.T_LD +s.P_RH +
    s.P_RH:s.T_LH + s.P_RD +
    site,
  data = sur_subset_GSP,
  family = "binomial",
  na.action = "na.fail"
)

n_cores <- detectCores()
n_cores
```

```
## [1] 20
```

```
cluster <- makeCluster(n_cores - 1)
registerDoParallel(cluster)
clusterExport(cluster, c("sur_subset_GSP")) ### replace with different subset (different global model)
clusterEvalQ(cluster, {library(lme4); library(MuMIn)})
```

```
## [[1]]
## [1] "MuMIn"      "lme4"       "Matrix"     "stats"      "graphics"   "grDevices"
## [7] "utils"      "datasets"   "methods"    "base"
##
## [[2]]
## [1] "MuMIn"      "lme4"       "Matrix"     "stats"      "graphics"   "grDevices"
## [7] "utils"      "datasets"   "methods"    "base"
##
## [[3]]
## [1] "MuMIn"      "lme4"       "Matrix"     "stats"      "graphics"   "grDevices"
## [7] "utils"      "datasets"   "methods"    "base"
##
## [[4]]
## [1] "MuMIn"      "lme4"       "Matrix"     "stats"      "graphics"   "grDevices"
## [7] "utils"      "datasets"   "methods"    "base"
##
## [[5]]
## [1] "MuMIn"      "lme4"       "Matrix"     "stats"      "graphics"   "grDevices"
## [7] "utils"      "datasets"   "methods"    "base"
##
## [[6]]
## [1] "MuMIn"      "lme4"       "Matrix"     "stats"      "graphics"   "grDevices"
## [7] "utils"      "datasets"   "methods"    "base"
##
## [[7]]
## [1] "MuMIn"      "lme4"       "Matrix"     "stats"      "graphics"   "grDevices"
## [7] "utils"      "datasets"   "methods"    "base"
```

```

##
## [[8]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[9]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[10]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[11]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[12]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[13]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[14]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[15]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[16]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[17]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[18]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[19]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"

```

```

sur_dredge_GSP <- MuMIn::dredge(
  GM_R_sur,
  cluster = cluster,
  trace   = 2
)

```

```
## Fixed term is "(Intercept)"
```

```
## |
```

```
sur_mod_GSP <- get.models(sur_dredge_GSP, 1)[[1]]  
summary(sur_mod_GSP)
```

```
##
```

```
## Call:
```

```
## glm(formula = sur0_1 ~ s.logsize0 + s.P_RD + s.P_RH + s.T_LA +  
##      s.T_LD + s.T_LH + s.TBF + s.TSF + site + s.P_RH:s.T_LH +  
##      s.TBF:s.TSF + 1, family = "binomial", data = sur_subset_GSP,  
##      na.action = "na.fail")
```

```
##
```

```
## Coefficients:
```

```
##      Estimate Std. Error z value Pr(>|z|)  
## (Intercept) -18.79222    3.99473  -4.704 2.55e-06 ***  
## s.logsize0   0.34950    0.07402   4.722 2.34e-06 ***  
## s.P_RD       3.32591    0.63574   5.232 1.68e-07 ***  
## s.P_RH       5.26207    0.95296   5.522 3.36e-08 ***  
## s.T_LA      -14.13410    2.51231  -5.626 1.85e-08 ***  
## s.T_LD       25.41684    4.47682   5.677 1.37e-08 ***  
## s.T_LH       4.37447    0.92750   4.716 2.40e-06 ***  
## s.TBF       -19.33423    3.66986  -5.268 1.38e-07 ***  
## s.TSF       -30.89807    5.84175  -5.289 1.23e-07 ***  
## siteGSP-LI   2.18423    0.42412   5.150 2.60e-07 ***  
## s.P_RH:s.T_LH 13.37197    2.34245   5.709 1.14e-08 ***  
## s.TBF:s.TSF  -22.23644    4.26464  -5.214 1.85e-07 ***
```

```
## ---
```

```
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
##
```

```
## (Dispersion parameter for binomial family taken to be 1)
```

```
##
```

```
##      Null deviance: 2085.9  on 3935  degrees of freedom
```

```
## Residual deviance: 1987.2  on 3924  degrees of freedom
```

```
## AIC: 2011.2
```

```
##
```

```
## Number of Fisher Scoring iterations: 6
```

```
stopCluster(cluster)
```

```
growth
```

```
gr_subset_GSP <-
```

```
  TBF_long_GSP[which(TBF_long_GSP$sur0_1 == 1),] %>%      ## make sure to subset data to only alive pla  
  filter_at(vars(logsize1, s.logsize0, s.TSF,s.sq.TSF, s.TBF, site, s.sq.T_RA,s.T_RA, s.P_RA, s.T_RC,  
                s.T_RD, s.P_RH), all_vars(!is.na(.)))
```

```
GM_R_gr <- lmer(  
  logsize1 ~ s.logsize0 + s.TSF * s.TBF + s.T_LA + s.sq.T_LA + s.P_RA + s.T_LA:s.P_RA +  
  s.T_LC + s.T_LD + s.P_RH + (1|site),  
  data = gr_subset_GSP,  
  na.action = "na.fail"
```

```

)

n_cores <- detectCores()
n_cores

## [1] 20

cluster <- makeCluster(n_cores - 1)
registerDoParallel(cluster)
clusterExport(cluster, c("gr_subset_GSP")) ### replace with different subset (different global models)
clusterEvalQ(cluster, {library(lme4); library(MuMIn)})

## [[1]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[2]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[3]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[4]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[5]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[6]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[7]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[8]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[9]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[10]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##

```

```
## [[11]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[12]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[13]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[14]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[15]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[16]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[17]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[18]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[19]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
```

```
gr_dredge_GSP <- MuMIn::dredge(
  GM_R_gr,
  cluster = cluster,
  trace   = 2
)
```

```
## Warning in .dredge.par(global.model = GM_R_gr, trace = 2, cluster = cluster):
## comparing models fitted by REML
```

```
## Fixed term is "(Intercept)"
```

```
##      |
```

```
growth_mod_GSP <- get.models(gr_dredge_GSP, 1)[[1]]
summary(growth_mod_GSP)
```

```

## Linear mixed model fit by REML ['lmerMod']
## Formula: logsize1 ~ s.logsize0 + s.P_RA + s.P_RH + s.sq.T_LA + s.T_LA +
##      s.T_LC + s.TBF + s.TSF + (1 | site) + s.P_RA:s.T_LA + s.TBF:s.TSF
##      Data: gr_subset_GSP
##
## REML criterion at convergence: 8258.2
##
## Scaled residuals:
##      Min       1Q   Median       3Q      Max
## -4.1682 -0.6497  0.0793  0.6770  3.1303
##
## Random effects:
##      Groups   Name      Variance Std.Dev.
##      site     (Intercept) 1.6474   1.2835
##      Residual             0.7748   0.8802
## Number of obs: 3182, groups: site, 2
##
## Fixed effects:
##              Estimate Std. Error t value
## (Intercept)    4.92085    0.91794   5.361
## s.logsize0      0.15394    0.01981   7.771
## s.P_RA          0.79375    0.13011   6.101
## s.P_RH         -1.35214    0.07760 -17.424
## s.sq.T_LA     -83.44195    6.28406 -13.278
## s.T_LA         84.90046    6.38301  13.301
## s.T_LC         -1.65792    0.12102 -13.700
## s.TBF           0.77001    0.12417   6.201
## s.TSF          -1.47607    0.15821  -9.330
## s.P_RA:s.T_LA  -0.54713    0.14178  -3.859
## s.TBF:s.TSF   -0.69814    0.32028  -2.180
##
## Correlation of Fixed Effects:
##              (Intr) s.lgs0 s.P_RA s.P_RH s..T_L s.T_LA s.T_LC s.TBF  s.TSF
## s.logsize0   -0.001
## s.P_RA        0.132 -0.008
## s.P_RH       -0.064 -0.294 -0.371
## s.sq.T_LA    -0.052 -0.288 -0.316  0.954
## s.T_LA       0.050  0.289  0.299 -0.952 -1.000
## s.T_LC      -0.070 -0.309 -0.502  0.934  0.915 -0.910
## s.TBF        0.141 -0.131  0.763 -0.329 -0.264  0.253 -0.294
## s.TSF        0.062 -0.272  0.115  0.262  0.252 -0.253  0.406  0.586
## s.P_RA:.T_L -0.130 -0.008 -0.986  0.306  0.267 -0.251  0.461 -0.737 -0.131
## s.TBF:s.TSF  0.137 -0.109  0.886 -0.157 -0.108  0.093 -0.236  0.859  0.500
##              s.P_RA:
## s.logsize0
## s.P_RA
## s.P_RH
## s.sq.T_LA
## s.T_LA
## s.T_LC
## s.TBF
## s.TSF
## s.P_RA:.T_L
## s.TBF:s.TSF -0.888

```

```
stopCluster(cluster)
```

```
prep
```

```
prep_subset_GSP <-  
  TBF_long_GSP %>%  
  filter_at(vars(prep1, s.logsize0, s.TSF, s.TBF,s.sq.TSF, site, s.T_RA,s.P_RA, s.T_RH, s.T_RC,  
    s.P_RH, ), all_vars(!is.na(.)))
```

```
GM_R_prep <- glm(  
  prep1 ~ s.logsize0 + s.TSF * s.TBF +  
    s.T_LA + s.sq.T_LA +s.P_RA+ s.sq.P_RA + s.T_LA:s.P_RA+  
    s.T_LH + s.P_RH + s.T_LH:s.P_RH + s.T_LC +  
    site,  
  data = prep_subset_GSP,  
  family = "binomial",  
  na.action = "na.fail"  
)
```

```
n_cores <- detectCores()  
n_cores
```

```
## [1] 20
```

```
cluster <- makeCluster(n_cores - 1)  
registerDoParallel(cluster)  
clusterExport(cluster, c("prep_subset_GSP")) ### replace with different subset (different global mode  
clusterEvalQ(cluster, {library(lme4); library(MuMIn)})
```

```
## [[1]]  
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"  
## [7] "utils"      "datasets"  "methods"   "base"      "graphics"  "grDevices"  
##  
## [[2]]  
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"  
## [7] "utils"      "datasets"  "methods"   "base"      "graphics"  "grDevices"  
##  
## [[3]]  
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"  
## [7] "utils"      "datasets"  "methods"   "base"      "graphics"  "grDevices"  
##  
## [[4]]  
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"  
## [7] "utils"      "datasets"  "methods"   "base"      "graphics"  "grDevices"  
##  
## [[5]]  
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"  
## [7] "utils"      "datasets"  "methods"   "base"      "graphics"  "grDevices"  
##  
## [[6]]  
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"  
## [7] "utils"      "datasets"  "methods"   "base"      "graphics"  "grDevices"
```

```

##
## [[7]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[8]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[9]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[10]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[11]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[12]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[13]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[14]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[15]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[16]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[17]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[18]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[19]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"

```

```

prep_dredge_GSP <- MuMin::dredge(
  GM_R_prep,
  cluster = cluster,
  trace   = 2
)

```

```
## Fixed term is "(Intercept)"
```

```
##      |
```

```

prep_mod_GSP <- get.models(prepare_dredge_GSP, 1)[[1]]
summary(prepare_mod_GSP)

```

```

##
## Call:
## glm(formula = prep1 ~ s.logsize0 + s.P_RA + s.P_RH + s.sq.P_RA +
##      s.sq.T_LA + s.T_LA + s.T_LH + s.TBF + s.TSF + site + 1, family = "binomial",
##      data = prep_subset_GSP, na.action = "na.fail")
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)    0.36088    0.45506   0.793   0.428
## s.logsize0      0.46376    0.08791   5.275 1.32e-07 ***
## s.P_RA         17.71190    2.19955   8.053 8.11e-16 ***
## s.P_RH         -2.43089    0.33025  -7.361 1.83e-13 ***
## s.sq.P_RA      -24.29691    2.87313  -8.457 < 2e-16 ***
## s.sq.T_LA      268.14861   36.69147   7.308 2.71e-13 ***
## s.T_LA        -261.04495   36.19164  -7.213 5.48e-13 ***
## s.T_LH         -21.52008    2.64097  -8.149 3.68e-16 ***
## s.TBF           7.98790    1.01883   7.840 4.50e-15 ***
## s.TSF          15.80181    2.03387   7.769 7.89e-15 ***
## siteGSP-LI     -1.21378    0.28828  -4.210 2.55e-05 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 1832.4  on 3642  degrees of freedom
## Residual deviance: 1522.2  on 3632  degrees of freedom
## AIC: 1544.2
##
## Number of Fisher Scoring iterations: 7

```

```
stopCluster(cluster)
```

```
crep
```

```

crep_subset_GSP <-
  TBF_long_GSP %>%
  filter_at(vars(logcrep1, s.logsize0, TSF, TBF, s.sq.TSF, site, s.T_RA, s.T_RH), all_vars(!is.na(.)))

```

```
GM_R_crep <- lmer(
  logcrep1 ~ s.logsize0 + s.TSF * s.TBF + s.T_LA + s.T_LH +
    (1 | site),
  data = crep_subset_GSP,
  na.action = "na.fail"
)

n_cores <- detectCores()
n_cores
```

```
## [1] 20
```

```
cluster <- makeCluster(n_cores - 1)
registerDoParallel(cluster)
clusterExport(cluster, c("crep_subset_GSP")) ### replace with different subset (different global mode
clusterEvalQ(cluster, {library(lme4); library(MuMIn)})
```

```
## [[1]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[2]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[3]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[4]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[5]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[6]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[7]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[8]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[9]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
```

```
##
## [[10]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[11]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[12]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[13]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[14]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[15]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[16]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[17]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[18]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[19]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
```

```
crep_dredge_GSP <- MuMIn::dredge(
  GM_R_crep,
  cluster = cluster,
  trace   = 2
)
```

```
## Warning in .dredge.par(global.model = GM_R_crep, trace = 2, cluster = cluster):
## comparing models fitted by REML
```

```
## Fixed term is "(Intercept)"
```

```
## |
```

```
|
```

```
crep_mod_GSP <- get.models(crep_dredge_GSP, 1)[[1]]

stopCluster(cluster)
```

recruitment

```
recruit_mod_GSP <- recruit_mod
```

variance in growth

```
growth_terms <- attr(terms(growth_mod_GSP), "term.labels")
int_terms <- grep(":", growth_terms, value = TRUE)
int_terms_broken <- unique(unlist(strsplit(int_terms, ":", fixed = TRUE)))
main_terms <- growth_terms[!grepl(":", growth_terms)]
logsize1 <- c("logsize1")
gr_all_terms <- unique(c(main_terms, int_terms_broken, logsize1))

Vargrowth_subset_GSP <-
  TBF_long_GSP[which(TBF_long_GSP$sur0_1 == 1),] %>%      ## make sure to subset data to only alive pla
  filter_at(vars( s.logsize0, TSF, TBF,logsize1, site), all_vars(!is.na(.)))

Vargrowth_subset_GSP$vargrowth <- NA
Vargrowth_subset_GSP$predgrowth <- NA

Vargrowth_subset_GSP$predgrowth <- predict(growth_mod_GSP,newdata=Vargrowth_subset_GSP)

Vargrowth_subset_GSP$vargrowth <-
  (Vargrowth_subset_GSP$predgrowth - # variance is equivalent to (predicted-expected)^2
   Vargrowth_subset_GSP$logsize1)^2

Vargrowth_subset_GSP$log.vargrowth <- log(Vargrowth_subset_GSP$vargrowth)

##### to prevent negative numbers later

GM_vargrowth_R <- lmer(log.vargrowth ~ s.logsize0 + s.T_LA+ s.TSF * s.TBF +
  s.P_RA + s.sq.P_RA + s.T_LC +s.T_LD +
  s.P_RD + s.P_RW
  + (1|site), #
  data = Vargrowth_subset_GSP, na.action = "na.fail")

n_cores <- detectCores()
cluster <- makeCluster(n_cores - 1)
registerDoParallel(cluster)
clusterExport(cluster, c("Vargrowth_subset_GSP")) ### replace with different subset (different global
clusterEvalQ(cluster, {library(lme4); library(MuMIn)})

## [[1]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[2]]
```

```

## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[3]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[4]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[5]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[6]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[7]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[8]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[9]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[10]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[11]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[12]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[13]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[14]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[15]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"

```

```
##
## [[16]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[17]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[18]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[19]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
```

```
vargrowth_dredge_GSP <- MuMIn::dredge(
  GM_vargrowth_R,
  cluster = cluster,
  trace = 2
)
```

```
## Warning in .dredge.par(global.model = GM_vargrowth_R, trace = 2, cluster =
## cluster): comparing models fitted by REML
```

```
## Fixed term is "(Intercept)"
```

```
##      |
```

```
vargrowth_mod_GSP <- get.models(vargrowth_dredge_GSP, 1)[[1]]

stopCluster(cluster)
```

```
TBF_long$logsize0 <- (log(TBF_long$size0+0.1))
TBF_long$s.logsize0 <- scale(TBF_long$logsize0)
TBF_long$s.TBF <- scale(TBF_long$TBF)
TBF_long$s.TSF <- scale(TBF_long$TSF)
TBF_long$sq.TSF <- (TBF_long$TSF)^2
TBF_long$s.sq.TSF <- scale(TBF_long$sq.TSF)
IPM_bounds <- range(TBF_long$logsize0, na.rm = TRUE) # our IPM will be based on logsize as a size metr
no_bins <- 100
bin_edges <- seq(IPM_bounds[1], IPM_bounds[2], length.out= no_bins+1)
bin_mids <- zoo::rollmean(bin_edges,2)

# define mean and sd of seedling sizes
rows_with_new <- TBF_long[grepl("\\bnew", TBF_long$comm1), ]
rows_with_new <- rows_with_new[!grepl("no new", rows_with_new$comm1), ] # delete "no new"
rows_with_new <- rows_with_new[!grepl("no news", rows_with_new$comm1), ] # delete "no new"
```

```

rows_with_new <- rows_with_new[!grepl("not possible to see new plants", rows_with_new$comm1), ]
rows_with_new <- rows_with_new[!grepl("No news", rows_with_new$comm1), ]
rows_with_new <- rows_with_new[!grepl("new lvs", rows_with_new$comm1), ]
rows_with_new <- rows_with_new[!grepl("new nail", rows_with_new$comm1), ]

mean_seedling_logsize <- mean(rows_with_new$logsize1, na.rm=T) # get the mean logsize of seedlings ###
var_seedling_logsize <- var(rows_with_new$logsize1, na.rm=T) # get the var in logsize of seedlings

### build empty matrices to store IPM output. Here, I'm interested in the effect of fire on lambda, so
possible_scenarios <-
  as.data.frame(matrix(NA, nrow= 22, ncol = 28))
names(possible_scenarios) <- c("FRI", "s.TBF", "s.T_LA", "s.T_LH",
  "s.sq.T_LH",
  "s.sq.P_LA",
  "s.T_LC",
  "s.T_LD",
  "s.P_LD",
  "s.sq.T_LA",
  "s.P_LA",
  "s.P_LW",
  "s.P_LH",
  "s.T_RA",
  "s.sq.T_RA",
  "s.P_RA",
  "s.T_RH",
  "s.P_RH",
  "s.T_RC",
  "s.T_RW",
  "s.T_RD",
  "s.sq.T_RC",
  "s.P_RW",
  "s.P_RD",
  "s.sq.T_RH",
  "s.sq.P_RA",
  "s.sq.P_RW",
  "s.sq.P_RD")

possible_scenarios$FRI <- rep(1:11, 2) # we want FRI to range from 1-11
possible_scenarios$s.TBF <- scale(rep(c(0, 10), each= 11), # raw TBF values
  center= attr(TBF_long$s.TBF, "scaled:center"),
  scale= attr(TBF_long$s.TBF, "scaled:scale")) # for each set of FRI's,

possible_scenarios[, c( "s.T_LA", "s.T_LH", "s.sq.T_LH", "s.sq.P_LA", "s.T_LC", "s.T_LD", "s.P_LD",
  "s.P_LW", "s.P_LH", "s.T_RA", "s.sq.T_RA", "s.P_RA", "s.T_RH", "s.P_RH",
  "s.sq.T_RC", "s.P_RW", "s.P_RD", "s.sq.T_RH", "s.sq.P_RA", "s.sq.P_RW", "s.sq.P_RD")]

my_sites <- unique(TBF_long_GSP$site)
no_reps <- 1000 # or somethign similar

##### extract model components for later predict() #####
sur_model_terms <- terms(sur_mod_GSP)
sur_term <- attr(sur_model_terms, "term.labels")

```

```

sur_term <- reformulate(sur_term)

gr_model_terms <- terms(growth_mod_GSP)
gr_term <- attr(gr_model_terms, "term.labels")
gr_term <- reformulate(gr_term)

prep_model_terms <- terms(prepare_mod_GSP)
prep_term <- attr(prepare_model_terms, "term.labels")
prep_term <- reformulate(prepare_term)

crep_model_terms <- terms(crep_mod_GSP)
crep_term <- attr(crep_model_terms, "term.labels")
crep_term <- reformulate(crep_term)

vargrowth_model_terms <- terms(vargrowth_mod_GSP)
vargrowth_term <- attr(vargrowth_model_terms, "term.labels")
vargrowth_term <- reformulate(vargrowth_term)

rec_model_terms <- terms(recruit_mod_GSP)
rec_term <- attr(rec_model_terms, "term.labels")
rec_term <- reformulate(rec_term)

```

```

all_pop_growth_rates_GSP_reg <- matrix(data= NA, nrow= dim(possible_scenarios)[1], ncol= no_reps)
n_cores <- detectCores()
cl <- makeCluster(n_cores - 1)
registerDoParallel(cl)
j_pop_gr_rate_GSP_reg <- rep(NA, dim(possible_scenarios)[1]) ### changed to no_scenarios

all_pop_growth_rates_GSP_reg <- foreach(
  j = 1:no_reps,
  .combine = cbind,
  .packages = c("MASS", "lme4", "popbio", "dplyr")
) %dopar% {
  rsur_coefs <- coefficients(sur_mod_GSP) # first, assign all "randomly selected" coefficients to the me
  rsur_coefs[!names(coefficients(sur_mod_GSP)) %in% paste("site", my_sites, sep="")] <- MASS::mvrnorm(mu= lme4::fixef(sur_mod_GSP), Sigma= vcov(sur_mod_GSP))
  #rsur_coefs <- lme4::fixef(sur_mod_GSP)
  #rsur_coefs <- MASS::mvrnorm(mu= lme4::fixef(sur_mod_GSP), Sigma= vcov(sur_mod_GSP))
  rgr_coefs <- lme4::fixef(growth_mod_GSP)
  rgr_coefs <- MASS::mvrnorm(mu= lme4::fixef(growth_mod_GSP), Sigma= vcov(growth_mod_GSP)) # no site e
  rprep_coefs <- coefficients(prepare_mod_GSP)
  rprep_coefs[!names(coefficients(prepare_mod_GSP)) %in% paste("site", my_sites, sep="")] <- MASS::mvrnorm(mu= lme4::fixef(prepare_mod_GSP), Sigma= vcov(prepare_mod_GSP))
  rcrep_coefs <- lme4::fixef(crep_mod_GSP)
  rcrep_coefs <- MASS::mvrnorm(mu= lme4::fixef(crep_mod_GSP), Sigma= vcov(crep_mod_GSP)) # no site e
  rvar_coefs <- lme4::fixef(vargrowth_mod_GSP)
  rvar_coefs <- MASS::mvrnorm(mu= lme4::fixef(vargrowth_mod_GSP), Sigma= vcov(vargrowth_mod_GSP)) # no site e
  rrec_coefs <- lme4::fixef(recruit_mod_GSP)
  rrec_coefs <- MASS::mvrnorm(mu= lme4::fixef(recruit_mod_GSP), Sigma= vcov(recruit_mod_GSP)) # no site e
  #rrec_coefs <- MASS::mvrnorm(mu= lme4::fixef(recruit_mod_GSP), Sigma= vcov(recruit_mod_GSP))

  for (i in 1: dim(possible_scenarios)[1]) {

    i_scenario <- possible_scenarios[i,] # which combination of FRI, TBF, and climate do I want to use

```

```

TSFs <- 0:(i_scenario$FRI-1) # TSF range from 0 to (FRI-1) such that an FRI of 1 will always have T
i_scenario <- i_scenario[rep(seq_len(nrow(i_scenario)), length(TSFs)), ]
i_scenario$TSF <- TSFs
i_scenario$s.TSF <- scale(TSFs, center= attr(TBF_long$s.TSF,"scaled:center"),
                          scale= attr(TBF_long$s.TSF,"scaled:scale"))
i_scenario$s.sq.TSF <- scale(TSFs^2, center= attr(TBF_long$s.sq.TSF,"scaled:center"),
                             scale= attr(TBF_long$s.sq.TSF,"scaled:scale"))

mx_prod <- diag(no_bins+1) # setting up an identity matrix before you enter the loop

for (ii in TSFs){
  i_ii_scenario <- i_scenario[which(i_scenario$TSF==ii),]
  data_for_prediction <- i_ii_scenario[rep(seq_len(nrow(i_ii_scenario)), length(bin_mids)), ]
  data_for_prediction$s.logsize0 <- scale(bin_mids, center= attr(TBF_long$s.logsize0,"scaled:center"),
                                         scale= attr(TBF_long$s.logsize0,"scaled:scale"))
  data_for_prediction_rec <- cbind(i_ii_scenario)

  predicted_sur <- predicted_gr <- predicted_pr <- predicted_cr <- predicted_vg <- matrix(data=NA, n
  predicted_rec <- rep(NA, length(my_sites))

  ### make empty mxes before entering the site loop

  predicted_sur_allsites <- predicted_gr_allsites <- predicted_pr_allsites <- predicted_cr_allsites <- pr
  predicted_rec_allsites <- matrix(NA, nrow = length(my_sites), ncol = 1, dimnames = list(my_sites,

  for (s in my_sites){ #looping through the sites
    s_data_for_prediction <- data_for_prediction
    s_data_for_prediction_rec <- data_for_prediction_rec
    s_data_for_prediction$site <- s
    s_data_for_prediction_rec$site <- s

    s_data_for_prediction$site <- factor(s_data_for_prediction$site, levels = my_sites) ## i
    s_data_for_prediction_rec$site <- factor(s_data_for_prediction_rec$site, levels = my_sites)

    s_data_for_prediction <- s_data_for_prediction %>% mutate(across(c("s.logsize0", "s.sq.TSF", "s

    m_sur <- model.matrix(sur_term, data=s_data_for_prediction)
    p2_sur <- rsur_coefs %*% t(m_sur) # check on whether this coef(survival_mod is a column or a row
    p2_sur <- plogis(p2_sur) ## transform to binomial
    predicted_sur_allsites[s,] <- p2_sur

    m_gr <- model.matrix(gr_term, data=s_data_for_prediction)
    p2_gr <- rgr_coefs %*% t(m_gr) # check on whether this coef(survival_mod is a column or a row!!
    predicted_gr_allsites[s,] <- p2_gr

    m_prep <- model.matrix(prepare_term, data=s_data_for_prediction)
    p2_prep <- rprep_coefs %*% t(m_prep) # check on whether this coef(survival_mod is a column or a
    p2_prep <- plogis(p2_prep) ## transform to binomial
    predicted_pr_allsites[s,] <- p2_prep

    m_crep <- model.matrix(crep_term, data=s_data_for_prediction)
    p2_crep <- rcrep_coefs %*% t(m_crep) # check on whether this coef(survival_mod is a column or a

```

```

predicted_cr_allsites[s,] <- p2_crep

m_vg <- model.matrix(vargrowth_term, data=s_data_for_prediction)
p2_vg <- rvar_coefs %*% t(m_vg) #
predicted_vg_allsites[s,] <- exp(p2_vg)

m_rec <- model.matrix(rec_term , data=s_data_for_prediction_rec)
p2_rec <- rrec_coefs %*% t(m_rec) # check on whether this coef(survival_mod is a column or a row)
predicted_rec_allsites[s,1] <- p2_rec*log(1.1) ##### account for composite rec variab

# for mixed models, the above functions will predict site-specific results (i.e., will incorporate
rm(s_data_for_prediction) # this part removes the site-specific dataframe used in this loop, so
}
predicted_sur <- apply(predicted_sur_allsites, MARGIN= 2, FUN= mean, na.rm=T) # average size-spec
predicted_gr <- apply(predicted_gr_allsites, MARGIN= 2, FUN= mean, na.rm=T)
predicted_pr <- apply(predicted_pr_allsites, MARGIN= 2, FUN= mean, na.rm=T)
predicted_cr <- apply(predicted_cr_allsites, MARGIN= 2, FUN= mean, na.rm=T)
predicted_vg <- apply(predicted_vg_allsites, MARGIN= 2, FUN= mean, na.rm=T)
predicted_vg[which(predicted_vg<0)] <- 0 ##### to prevent negative variance
predicted_rec <- mean(predicted_rec_allsites, na.rm=T)

gmx <- matrix(NA, length(bin_mids),length(bin_mids)) ## create empty gmx before entering the no_b

# make the TSF-specific matrix
for (ss in 1:no_bins) {
  growcdf <- pnorm(bin_edges,predicted_gr[ss],sqrt(predicted_vg[ss])) # Yes. Vargrowth=(pred - ob
  grows <- growcdf[2:length(bin_edges)]-growcdf[1:(length(bin_edges)-1)]
  if(sum(grows)>0){grows <- grows/sum(grows)}
  gmx[,ss] <- grows
  } else if (sum(grows)==0 & predicted_gr[ss]< min(bin_edges)) {
    gmx[,ss] <- c(1, rep(0, length(bin_mids)-1))
  } else if (sum(grows)==0 & predicted_gr[ss]> max(bin_edges)) {
    gmx[,ss] <- c(rep(0, length(bin_mids)-1), 1) } else { # deals with complete eviction-- OUT D
    gmx[,ss] <- NA}
  # this if statement breaks the code (puts NA's into the matrix) if the sum of the PDF is zero (
} # end ss loop

# make the surv*growth mx
survgmx <- gmx*t(matrix( rep(predicted_sur,no_bins),no_bins)) # Yes, this is doing element-wise m
repro <- predicted_sur*predicted_pr*predicted_cr # prob that you survive, reproduce, then number

mx <- matrix(0, no_bins+1, no_bins+1)
mx[2:(no_bins+1), 2:(no_bins+1)] = survgmx
mx[1,2:(no_bins+1)] = repro # FRUIT ROW

sdlng_cdf <- pnorm(bin_edges,mean_seedling_logsize,sqrt(var_seedling_logsize))
sdlng_pdf <- sdlng_cdf[2:length(bin_edges)]-sdlng_cdf[1:(length(bin_edges)-1)]
if(sum(sdlng_pdf)>0){
  sdlng_pdf <- sdlng_pdf/sum(sdlng_pdf)} else {
  sdlng_pdf <- NA}

mx[2:(no_bins+1),1] = predicted_rec *sdlng_pdf

```

```

    mx_prod <- mx_prod %*% mx # in the first iteration of the loop, mx just becomes mx_prod

  } # closes ii (TSFs) loop

  j_pop_gr_rate_GSP_reg[i] <- (popbio::lambda(mx_prod))^(1/length(TSFs))
  # multiply them together to get a matrix, etc.
}
j_pop_gr_rate_GSP_reg
}

```

2.2.2 GSP IPM

```

all_pop_growth_rates_GSP_reg <- cbind(possible_scenarios, all_pop_growth_rates_GSP_reg)
names(all_pop_growth_rates_GSP_reg) <- c(names(possible_scenarios), paste("rep", 1:no_reps, sep = ""))
rep_cols <- grep("^rep", names(all_pop_growth_rates_GSP_reg), value = TRUE)

mean_lambda_across_scenarios_GSP_reg <- all_pop_growth_rates_GSP_reg %>%
  rowwise() %>%
  mutate(
    mean_lambda = mean(c_across(all_of(rep_cols)), na.rm = TRUE),
    lower_bound = quantile(c_across(all_of(rep_cols)), 0.025, na.rm = TRUE),
    upper_bound = quantile(c_across(all_of(rep_cols)), 0.975, na.rm = TRUE)
  ) %>%
  dplyr::select(FRI,s.TBF,mean_lambda,lower_bound, upper_bound) %>%
  mutate(TBF=NA) %>%
  ungroup()

mean_lambda_across_scenarios_GSP_reg$TBF[1:11] <- "Short TBF"
mean_lambda_across_scenarios_GSP_reg$TBF[12:22] <- "Long TBF"

#####. graph
#####

plt6<- ggplot(data= mean_lambda_across_scenarios_GSP_reg, aes(x= FRI, y= mean_lambda)) +
  geom_line(aes(color= TBF)) +
  geom_ribbon(aes(ymin = mean_lambda_across_scenarios_GSP_reg$lower_bound, ymax = mean_lambda_across_scenarios_GSP_reg$upper_bound, fill = TBF)) +
  #ylim(0,1) +
  #xlim(c(0, 12)) +
  labs(y= "Growth Rate", x= "Fire Return Interval",color = NULL, fill = NULL) +
  theme_bw() + theme(legend.position='none') +
  theme(legend.position = "right") +
  scale_x_continuous(breaks = 0:10) +
  theme(text = element_text(size = 16))
plt6

```

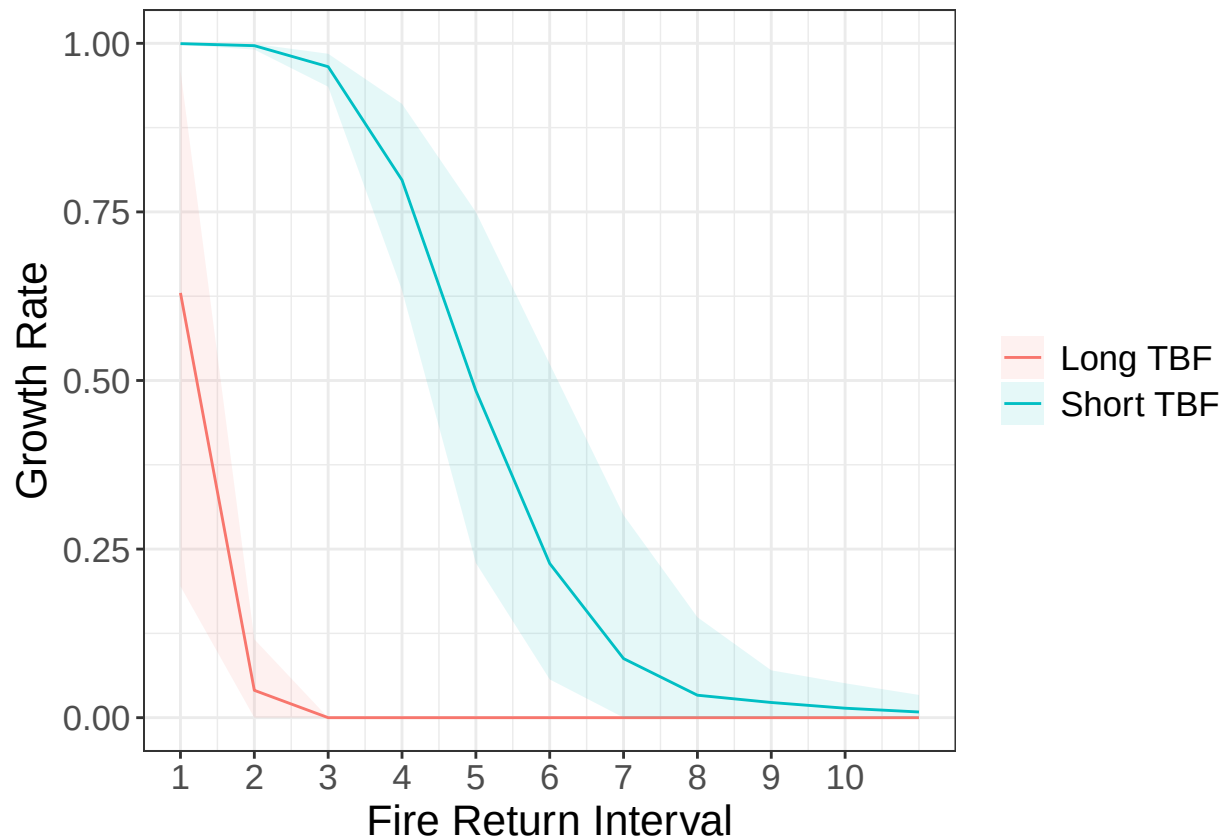
2.2.3 graph

```

## Warning: Use of 'mean_lambda_across_scenarios_GSP_reg$lower_bound' is discouraged.
## i Use 'lower_bound' instead.

```

```
## Warning: Use of 'mean_lambda_across_scenarios_GSP_reg$upper_bound' is discouraged.
## i Use 'upper_bound' instead.
```



2.3 Fort Bragg

The Fort Bragg region contains B1 and B2. These two populations are in only inland populations.

```
TBF_long_FB <- TBF_long %>%
  filter(site=="B1"|site=="B2")
```

2.3.1 refit vital rate survival

```
sur_subset_FB <-
  TBF_long_FB %>%
  filter_at(vars(sur0_1, s.logsize0, s.TSF,s.TBF, site), all_vars(!is.na(.)))

GM_R_sur <- glm(
  sur0_1 ~ s.logsize0 + s.TSF * s.TBF +
    s.T_LA + s.T_LH + s.sq.T_LH + s.T_LD +s.P_RH +
    s.P_RH:s.T_LH + s.P_RD +
    site,
  data = sur_subset_FB,
  family = "binomial",
  na.action = "na.fail"
```

```

)

n_cores <- detectCores()
n_cores

## [1] 20

cluster <- makeCluster(n_cores - 1)
registerDoParallel(cluster)
clusterExport(cluster, c("sur_subset_FB")) ### replace with different subset (different global models)
clusterEvalQ(cluster, {library(lme4); library(MuMIn)})

## [[1]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[2]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[3]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[4]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[5]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[6]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[7]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[8]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[9]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[10]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##

```

```
## [[11]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"    "base"
##
## [[12]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"    "base"
##
## [[13]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"    "base"
##
## [[14]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"    "base"
##
## [[15]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"    "base"
##
## [[16]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"    "base"
##
## [[17]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"    "base"
##
## [[18]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"    "base"
##
## [[19]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"    "base"
```

```
sur_dredge_FB <- MuMIn::dredge(
  GM_R_sur,
  cluster = cluster,
  trace    = 2
)
```

```
## Fixed term is "(Intercept)"
```

```
## |
```

```
sur_mod_FB <- get.models(sur_dredge_FB, 1)[[1]]
summary(sur_mod_FB)
```

```
##
## Call:
## glm(formula = sur0_1 ~ s.logsize0 + s.P_RH + s.sq.T_LH + s.T_LH +
```

```
##      s.TBF + s.TSF + site + s.TBF:s.TSF + 1, family = "binomial",
##      data = sur_subset_FB, na.action = "na.fail")
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)   3.20818    0.38548   8.322 < 2e-16 ***
## s.logsize0     0.50213    0.06170   8.138 4.02e-16 ***
## s.P_RH         0.28639    0.08121   3.527 0.000421 ***
## s.sq.T_LH     -32.40671    8.06614  -4.018 5.88e-05 ***
## s.T_LH        31.48427    7.69473   4.092 4.28e-05 ***
## s.TBF          0.46608    0.10994   4.239 2.24e-05 ***
## s.TSF         -2.45316    0.26377  -9.300 < 2e-16 ***
## siteB2        -1.48512    0.21192  -7.008 2.42e-12 ***
## s.TBF:s.TSF    0.46828    0.24556   1.907 0.056521 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 1804.0  on 4181  degrees of freedom
## Residual deviance: 1575.6  on 4173  degrees of freedom
## AIC: 1593.6
##
## Number of Fisher Scoring iterations: 6
```

```
stopCluster(cluster)
```

growth

```
gr_subset_FB <-
  TBF_long_FB[which(TBF_long_FB$sur0_1 == 1),] %>%      ## make sure to subset data to only alive plants
  filter_at(vars(logsize1, s.logsize0, s.TSF,s.sq.TSF, s.TBF, site, s.sq.T_RA,s.T_RA, s.P_RA, s.T_RC,
                  s.T_RD, s.P_RH), all_vars(!is.na(.)))
GM_R_gr <- lmer(
  logsize1 ~ s.logsize0 + s.TSF * s.TBF + s.T_LA + s.sq.T_LA + s.P_RA + s.T_LA:s.P_RA +
  s.T_LC + s.T_LD + s.P_RH + (1|site),
  data = gr_subset_FB,
  na.action = "na.fail"
)

n_cores <- detectCores()
n_cores
```

```
## [1] 20
```

```
cluster <- makeCluster(n_cores - 1)
registerDoParallel(cluster)
clusterExport(cluster, c("gr_subset_FB"))  ### replace with different subset (different global models)
clusterEvalQ(cluster, {library(lme4); library(MuMIn)})
```

```
## [[1]]
## [1] "MuMIn"      "lme4"       "Matrix"     "stats"      "graphics"   "grDevices"
```

```

## [7] "utils"      "datasets"  "methods"   "base"
##
## [[2]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[3]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[4]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[5]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[6]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[7]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[8]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[9]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[10]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[11]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[12]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[13]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[14]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##

```

```
## [[15]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[16]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[17]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[18]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[19]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
```

```
gr_dredge_FB <- MuMIn::dredge(
  GM_R_gr,
  cluster = cluster,
  trace   = 2
)
```

```
## Warning in .dredge.par(global.model = GM_R_gr, trace = 2, cluster = cluster):
## comparing models fitted by REML
```

```
## Fixed term is "(Intercept)"
```

```
##      |
```

```
growth_mod_FB <- get.models(gr_dredge_FB, 1)[[1]]
summary(growth_mod_FB)
```

```
## Linear mixed model fit by REML ['lmerMod']
## Formula: logsize1 ~ s.logsize0 + s.P_RA + s.P_RH + s.sq.T_LA + s.T_LA +
##          s.TBF + s.TSF + (1 | site) + s.TBF:s.TSF
## Data: gr_subset_FB
##
## REML criterion at convergence: 9225.4
##
## Scaled residuals:
##      Min       1Q   Median       3Q      Max
## -8.2174 -0.4747  0.1173  0.6253  3.6274
##
## Random effects:
##  Groups   Name                Variance Std.Dev.
##  site      (Intercept)  0.2951     0.5432
##  Residual                    0.8818     0.9390
## Number of obs: 3389, groups: site, 2
```

```
##
## Fixed effects:
##           Estimate Std. Error t value
## (Intercept)  5.3942708  0.5791124   9.315
## s.logsize0    0.2339760  0.0168993  13.845
## s.P_RA       -0.3319251  0.0393190  -8.442
## s.P_RH        0.3771363  0.0452223   8.340
## s.sq.T_LA     1.4084776  5.3587028   0.263
## s.T_LA       -1.0217635  5.0393071  -0.203
## s.TBF        -0.0005588  0.0291163  -0.019
## s.TSF         0.0229864  0.0689716   0.333
## s.TBF:s.TSF   0.2389158  0.0561723   4.253
##
## Correlation of Fixed Effects:
##           (Intr) s.lgs0 s.P_RA s.P_RH s..T_L s.T_LA s.TBF  s.TSF
## s.logsize0    0.059
## s.P_RA       -0.011  0.083
## s.P_RH       -0.019 -0.093 -0.878
## s.sq.T_LA     0.733  0.095  0.015 -0.073
## s.T_LA       -0.732 -0.096 -0.018  0.076 -1.000
## s.TBF        -0.220  0.117 -0.103  0.177 -0.306  0.307
## s.TSF        -0.283 -0.035  0.655 -0.631 -0.405  0.404 -0.007
## s.TBF:s.TSF   0.107  0.082  0.398 -0.429  0.178 -0.180  0.341  0.140
```

```
stopCluster(cluster)
```

```
prep
```

```
prep_subset_FB <-
  TBF_long_FB %>%
  filter_at(vars(prep1, s.logsize0, s.TSF, s.TBF,s.sq.TSF, site, s.T_RA,s.P_RA, s.T_RH, s.T_RC,
    s.P_RH, ), all_vars(!is.na(.)))

GM_R_prep <- glm(
  prep1 ~ s.logsize0 + s.TSF * s.TBF +
    s.T_LA + s.sq.T_LA +s.P_RA+ s.sq.P_RA + s.T_LA:s.P_RA+
    s.T_LH + s.P_RH + s.T_LH:s.P_RH + s.T_LC +
    site,
  data = prep_subset_FB,
  family = "binomial",
  na.action = "na.fail"
)

n_cores <- detectCores()
n_cores
```

```
## [1] 20
```

```
cluster <- makeCluster(n_cores - 1)
registerDoParallel(cluster)
clusterExport(cluster, c("prep_subset_FB")) ### replace with different subset (different global model)
clusterEvalQ(cluster, {library(lme4); library(MuMIn)})
```

```

## [[1]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[2]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[3]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[4]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[5]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[6]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[7]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[8]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[9]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[10]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[11]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[12]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[13]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[14]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"

```

```
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[15]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[16]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[17]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[18]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[19]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
```

```
prep_dredge_FB <- MuMIn::dredge(
  GM_R_prep,
  cluster = cluster,
  trace   = 2
)
```

```
## Fixed term is "(Intercept)"
```

```
##      |
```

```
prep_mod_FB <- get.models(prepared_dredge_FB, 1)[[1]]
summary(prep_mod_FB)
```

```
##
## Call:
## glm(formula = prep1 ~ s.logsize0 + s.P_RA + s.P_RH + s.sq.P_RA +
##      s.sq.T_LA + s.T_LA + s.T_LH + s.TBF + s.TSF + site + s.P_RA:s.T_LA +
##      s.P_RH:s.T_LH + s.TBF:s.TSF + 1, family = "binomial", data = prep_subset_FB,
##      na.action = "na.fail")
##
## Coefficients:
##      Estimate Std. Error z value Pr(>|z|)
## (Intercept)    4.61086    1.99690   2.309 0.020943 *
## s.logsize0      0.52234    0.05691   9.179 < 2e-16 ***
## s.P_RA         13.54670    1.62062   8.359 < 2e-16 ***
## s.P_RH         -1.20324    0.59688  -2.016 0.043812 *
## s.sq.P_RA     -15.79668    1.82477  -8.657 < 2e-16 ***
## s.sq.T_LA      94.51765   24.83228   3.806 0.000141 ***
## s.T_LA       -88.94687   23.28229  -3.820 0.000133 ***
## s.T_LH        -0.88755    0.29861  -2.972 0.002956 **
```

```
## s.TBF          -0.01351    0.11148  -0.121  0.903576
## s.TSF          -0.54187    0.25552  -2.121  0.033950 *
## siteB2         -0.51527    0.22020  -2.340  0.019286 *
## s.P_RA:s.T_LA  -0.51140    0.30670  -1.667  0.095429 .
## s.P_RH:s.T_LH  -1.34220    0.51898  -2.586  0.009703 **
## s.TBF:s.TSF     2.56374    0.38843    6.600  4.11e-11 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
## Null deviance: 3899.1 on 3947 degrees of freedom
## Residual deviance: 3454.2 on 3934 degrees of freedom
## AIC: 3482.2
##
## Number of Fisher Scoring iterations: 7
```

```
stopCluster(cluster)
```

```
crep
```

```
crep_subset_FB <-
  TBF_long_FB %>%
  filter_at(vars(logcrep1, s.logsize0, TSF, TBF,s.sq.TSF, site, s.T_RA, s.T_RH), all_vars(!is.na(.)))

GM_R_crep <- lmer(
  logcrep1 ~ s.logsize0 + s.TSF * s.TBF + s.T_LA + s.T_LH +
    (1 | site),
  data = crep_subset_FB,
  na.action = "na.fail"
)

n_cores <- detectCores()
n_cores
```

```
## [1] 20
```

```
cluster <- makeCluster(n_cores - 1)
registerDoParallel(cluster)
clusterExport(cluster, c("crep_subset_FB")) ### replace with different subset (different global model)
clusterEvalQ(cluster, {library(lme4); library(MuMIn)})
```

```
## [[1]]
## [1] "MuMIn"      "lme4"       "Matrix"     "stats"      "graphics"   "grDevices"
## [7] "utils"      "datasets"   "methods"    "base"
##
## [[2]]
## [1] "MuMIn"      "lme4"       "Matrix"     "stats"      "graphics"   "grDevices"
## [7] "utils"      "datasets"   "methods"    "base"
##
## [[3]]
## [1] "MuMIn"      "lme4"       "Matrix"     "stats"      "graphics"   "grDevices"
```

```

## [7] "utils"      "datasets"  "methods"   "base"
##
## [[4]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[5]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[6]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[7]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[8]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[9]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[10]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[11]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[12]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[13]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[14]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[15]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[16]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##

```

```
## [[17]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"    "base"
##
## [[18]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"    "base"
##
## [[19]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"    "base"
```

```
crep_dredge_FB <- MuMIn::dredge(
  GM_R_crep,
  cluster = cluster,
  trace = 2
)
```

```
## Warning in .dredge.par(global.model = GM_R_crep, trace = 2, cluster = cluster):
## comparing models fitted by REML
```

```
## Fixed term is "(Intercept)"
```

```
##      |
```

```
crep_mod_FB <- get.models(crep_dredge_FB, 1)[[1]]

stopCluster(cluster)
```

```
recruitment
```

```
recruit_mod_FB <- recruit_mod
```

```
variance in growth
```

```
growth_terms <- attr(terms(growth_mod_FB), "term.labels")
int_terms <- grep(":", growth_terms, value = TRUE)
int_terms_broken <- unique(unlist(strsplit(int_terms, ":", fixed = TRUE)))
main_terms <- growth_terms[!grepl(":", growth_terms)]
logsize1 <- c("logsize1")
gr_all_terms <- unique(c(main_terms, int_terms_broken, logsize1))
```

```
Vargrowth_subset_FB <-
  TBF_long_FB[which(TBF_long_FB$sur0_1 == 1),] %>%      ## make sure to subset data to only alive plants
  filter_at(vars( s.logsize0, TSF, TBF,logsize1, site), all_vars(!is.na(.)))
```

```
Vargrowth_subset_FB$vargrowth <- NA
Vargrowth_subset_FB$predgrowth <- NA
```

```
Vargrowth_subset_FB$predgrowth <- predict(growth_mod_FB,newdata=Vargrowth_subset_FB)
```

```

Vargrowth_subset_FB$vargrowth <-
  (Vargrowth_subset_FB$predgrowth - # variance is equivalent to (predicted-expected)^2
   Vargrowth_subset_FB$logsize1)^2

Vargrowth_subset_FB$log.vargrowth <- log(Vargrowth_subset_FB$vargrowth)

##### to prevent negative numbers later

GM_vargrowth_R <- lmer(log.vargrowth ~ s.logsize0 + s.T_LA+ s.TSF * s.TBF +
  s.P_RA + s.sq.P_RA + s.T_LC +s.T_LD +
  s.P_RD + s.P_RW
  + (1|site), #
  data = Vargrowth_subset_FB, na.action = "na.fail")

```

```
## boundary (singular) fit: see help('isSingular')
```

```

n_cores <- detectCores()
cluster <- makeCluster(n_cores - 1)
registerDoParallel(cluster)
clusterExport(cluster, c("Vargrowth_subset_FB")) ### replace with different subset (different global
clusterEvalQ(cluster, {library(lme4); library(MuMIn)})

```

```

## [[1]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[2]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[3]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[4]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[5]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[6]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[7]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[8]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"

```

```

## [7] "utils"      "datasets"  "methods"   "base"
##
## [[9]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[10]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[11]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[12]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[13]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[14]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[15]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[16]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[17]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[18]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"
##
## [[19]]
## [1] "MuMIn"      "lme4"      "Matrix"    "stats"     "graphics"  "grDevices"
## [7] "utils"      "datasets"  "methods"   "base"

```

```

vargrowth_dredge_FB <- MuMIn::dredge(
  GM_vargrowth_R,
  cluster = cluster,
  trace   = 2
)

```

```

## Warning in .dredge.par(global.model = GM_vargrowth_R, trace = 2, cluster =
## cluster): comparing models fitted by REML

```

```
## Fixed term is "(Intercept)"
```

```
## |
```

```
|
```

```
vargrowth_mod_FB <- get.models(vargrowth_dredge_FB, 1)[[1]]
```

```
stopCluster(cluster)
```

```
TBF_long$logsize0 <- (log(TBF_long$size0+0.1))
TBF_long$s.logsize0 <- scale(TBF_long$logsize0)
TBF_long$s.TBF <- scale(TBF_long$TBF)
TBF_long$s.TSF <- scale(TBF_long$TSF)
TBF_long$sq.TSF <- (TBF_long$TSF)^2
TBF_long$s.sq.TSF <- scale(TBF_long$sq.TSF)
IPM_bounds <- range(TBF_long$logsize0, na.rm = TRUE) # our IPM will be based on logsize as a size metr
no_bins <- 100
bin_edges <- seq(IPM_bounds[1], IPM_bounds[2], length.out= no_bins+1)
bin_mids <- zoo::rollmean(bin_edges,2)
```

```
# define mean and sd of seedling sizes
```

```
rows_with_new <- TBF_long[grepl("\\bnew", TBF_long$comm1), ]
rows_with_new <- rows_with_new[!grepl("no new", rows_with_new$comm1), ] # delete "no new"
rows_with_new <- rows_with_new[!grepl("no news", rows_with_new$comm1), ] # delete "no new"
rows_with_new <- rows_with_new[!grepl("not possible to see new plants", rows_with_new$comm1), ]
rows_with_new <- rows_with_new[!grepl("No news", rows_with_new$comm1), ]
rows_with_new <- rows_with_new[!grepl("new lvs", rows_with_new$comm1), ]
rows_with_new <- rows_with_new[!grepl("new nail", rows_with_new$comm1), ]
```

```
mean_seedling_logsize <- mean(rows_with_new$logsize1, na.rm=T) # get the mean logsize of seedlings ###
```

```
var_seedling_logsize <- var(rows_with_new$logsize1, na.rm=T) # get the var in logsize of seedlings
```

```
### build empty matrices to store IPM output. Here, I'm interested in the effect of fire on lambda, so
```

```
possible_scenarios <-
```

```
  as.data.frame(matrix(NA, nrow= 22, ncol = 28))
names(possible_scenarios) <- c("FRI", "s.TBF", "s.T_LA", "s.T_LH",
  "s.sq.T_LH",
  "s.sq.P_LA",
  "s.T_LC",
  "s.T_LD",
  "s.P_LD",
  "s.sq.T_LA",
  "s.P_LA",
  "s.P_LW",
  "s.P_LH",
  "s.T_RA",
  "s.sq.T_RA",
  "s.P_RA",
  "s.T_RH",
  "s.P_RH",
  "s.T_RC",
```

```

"s.T_RW",
"s.T_RD",
"s.sq.T_RC",
"s.P_RW",
"s.P_RD",
"s.sq.T_RH",
"s.sq.P_RA",
"s.sq.P_RW",
"s.sq.P_RD")

possible_scenarios$FRI <- rep(1:11, 2) # we want FRI to range from 1-11
possible_scenarios$s.TBF <- scale(rep(c(0, 10), each= 11) ,# raw TBF values
                                center= attr(TBF_long$s.TBF,"scaled:center"),
                                scale= attr(TBF_long$s.TBF,"scaled:scale")) # for each set of FRI's, i

possible_scenarios[, c( "s.T_LA", "s.T_LH", "s.sq.T_LH", "s.sq.P_LA", "s.T_LC", "s.T_LD", "s.P_L",
                        "s.P_LW", "s.P_LH", "s.T_RA", "s.sq.T_RA", "s.P_RA", "s.T_RH", "s.P_RH",
                        "s.sq.T_RC","s.P_RW","s.P_RD", "s.sq.T_RH", "s.sq.P_RA", "s.sq.P_RW", "s.sq

my_sites <- unique(TBF_long_FB$site)
no_reps <- 1000 # or somethign similar

##### extract model components for later predict() #####
sur_model_terms <- terms(sur_mod_FB)
sur_term <- attr(sur_model_terms, "term.labels")
sur_term <- reformulate(sur_term)

gr_model_terms <- terms(growth_mod_FB)
gr_term <- attr(gr_model_terms, "term.labels")
gr_term <- reformulate(gr_term)

prep_model_terms <- terms(prepare_mod_FB)
prep_term <- attr(prepare_model_terms, "term.labels")
prep_term <- reformulate(prep_term)

crep_model_terms <- terms(crep_mod_FB)
crep_term <- attr(crep_model_terms, "term.labels")
crep_term <- reformulate(crep_term)

vargrowth_model_terms <- terms(vargrowth_mod_FB)
vargrowth_term <- attr(vargrowth_model_terms, "term.labels")
vargrowth_term <- reformulate(vargrowth_term)

rec_model_terms <- terms(recruit_mod_FB)
rec_term <- attr(rec_model_terms, "term.labels")
rec_term <- reformulate(rec_term)

all_pop_growth_rates_FB_reg <- matrix(data= NA, nrow= dim(possible_scenarios)[1], ncol= no_reps)
n_cores <- detectCores()
cl <- makeCluster(n_cores - 1)
registerDoParallel(cl)

```

```

j_pop_gr_rate_FB_reg <- rep(NA, dim(possible_scenarios)[1]) ### changed to no_scenarios

all_pop_growth_rates_FB_reg <- foreach(
  j = 1:no_reps,
  .combine = cbind,
  .packages = c("MASS", "lme4", "popbio", "dplyr")
) %dopar% {
  rsur_coefs <- coefficients(sur_mod_FB) # first, assign all "randomly selected" coefficients to the mean
  rsur_coefs[!names(coefficients(sur_mod_FB)) %in% paste("site", my_sites, sep="")] <- MASS::mvrnorm(mu=
#rsur_coefs <- lme4::fixef(survival_mod)
#rsur_coefs <- MASS::mvrnorm(mu= lme4::fixef(survival_mod) , Sigma = vcov(survival_mod))
  rgr_coefs <- lme4::fixef(growth_mod_FB)
  rgr_coefs <- MASS::mvrnorm(mu= lme4::fixef(growth_mod_FB) , Sigma = vcov(growth_mod_FB)) # no site effects
  rprep_coefs <- coefficients(prepare_mod_FB)
  rprep_coefs[!names(coefficients(prepare_mod_FB)) %in% paste("site", my_sites, sep="")] <- MASS::mvrnorm(mu=
  rcrep_coefs <- lme4::fixef(crep_mod_FB)
  rcrep_coefs <- MASS::mvrnorm(mu= lme4::fixef(crep_mod_FB) , Sigma = vcov(crep_mod_FB)) # no site effects
  rvar_coefs <- lme4::fixef(vargrowth_mod_FB)
  rvar_coefs <- MASS::mvrnorm(mu= lme4::fixef(vargrowth_mod_FB) , Sigma = vcov(vargrowth_mod_FB)) # no site effects
  rrec_coefs <- lme4::fixef(recruit_mod_FB)
  rrec_coefs <- MASS::mvrnorm(mu= lme4::fixef(recruit_mod_FB) , Sigma = vcov(recruit_mod_FB)) # no site effects
#rrec_coefs <- MASS::mvrnorm(mu= lme4::fixef(recruit_mod) , Sigma = vcov(recruit_mod))

  for (i in 1: dim(possible_scenarios)[1]) {

    i_scenario <- possible_scenarios[i,] # which combination of FRI, TBF, and climate do I want to use

    TSFs <- 0:(i_scenario$FRI-1) # TSF range from 0 to (FRI-1) such that an FRI of 1 will always have T
    i_scenario <- i_scenario[rep(seq_len(nrow(i_scenario)), length(TSFs)), ]
    i_scenario$TSF <- TSFs
    i_scenario$s.TSF <- scale(TSFs, center= attr(TBF_long$s.TSF,"scaled:center"),
                             scale= attr(TBF_long$s.TSF,"scaled:scale"))
    i_scenario$s.sq.TSF <- scale(TSFs^2, center= attr(TBF_long$s.sq.TSF,"scaled:center"),
                                scale= attr(TBF_long$s.sq.TSF,"scaled:scale"))

    mx_prod <- diag(no_bins+1) # setting up an identity matrix before you enter the loop

    for (ii in TSFs){
      i_ii_scenario <- i_scenario[which(i_scenario$TSF==ii),]
      data_for_prediction <- i_ii_scenario[rep(seq_len(nrow(i_ii_scenario)), length(bin_mids)), ]
      data_for_prediction$s.logsize0 <- scale(bin_mids, center= attr(TBF_long$s.logsize0,"scaled:center"),
                                             scale= attr(TBF_long$s.logsize0,"scaled:scale"))
      data_for_prediction_rec <- cbind(i_ii_scenario)

      predicted_sur <- predicted_gr <- predicted_pr <- predicted_cr <- predicted_vg <- matrix(data=NA, nrow = length(my_sites), ncol = 1, dimnames = list(my_sites, "predicted_sur"))
      predicted_rec <- rep(NA, length(my_sites))

      ### make empty mxes before entering the site loop

      predicted_sur_allsites <- predicted_gr_allsites <- predicted_pr_allsites <- predicted_cr_allsites <- predicted_vg_allsites <- matrix(NA, nrow = length(my_sites), ncol = 1, dimnames = list(my_sites, "predicted_sur"))
      predicted_rec_allsites <- matrix(NA, nrow = length(my_sites), ncol = 1, dimnames = list(my_sites, "predicted_rec"))
    }
  }
}

```

```

for (s in my_sites){ #looping through the sites
  s_data_for_prediction <- data_for_prediction
  s_data_for_prediction_rec <- data_for_prediction_rec
  s_data_for_prediction$site <- s
  s_data_for_prediction_rec$site <- s

  s_data_for_prediction$site <- factor(s_data_for_prediction$site, levels = my_sites) ## i
  s_data_for_prediction_rec$site <- factor(s_data_for_prediction_rec$site, levels = my_sites)

  s_data_for_prediction <- s_data_for_prediction %>% mutate(across(c("s.logsize0", "s.sq.TSF", "s

  m_sur <- model.matrix(sur_term ,data=s_data_for_prediction)
  p2_sur <- rsur_coefs %*% t(m_sur) # check on whether this coef(survival_mod is a column or a row
  p2_sur <- plogis(p2_sur) ## transform to binomial
  predicted_sur_allsites[s,] <- p2_sur

  m_gr <- model.matrix(gr_term, data=s_data_for_prediction)
  p2_gr <- rgr_coefs %*% t(m_gr) # check on whether this coef(survival_mod is a column or a row!
  predicted_gr_allsites[s,] <- p2_gr

  m_prep <- model.matrix(prepare_term,data=s_data_for_prediction)
  p2_prep <- rprep_coefs %*% t(m_prep) # check on whether this coef(survival_mod is a column or a
  p2_prep <- plogis(p2_prep) ## transform to binomial
  predicted_pr_allsites[s,] <- p2_prep

  m_crep <- model.matrix(crep_term ,data=s_data_for_prediction)
  p2_crep <- rcrep_coefs %*% t(m_crep) # check on whether this coef(survival_mod is a column or a
  predicted_cr_allsites[s,] <- p2_crep

  m_vg <- model.matrix(vargrowth_term,data=s_data_for_prediction)
  p2_vg <- rvar_coefs %*% t(m_vg) #
  predicted_vg_allsites[s,] <- exp(p2_vg)

  m_rec <- model.matrix(rec_term , data=s_data_for_prediction_rec)
  p2_rec <- rrec_coefs %*% t(m_rec) # check on whether this coef(survival_mod is a column or a row
  predicted_rec_allsites[s,1] <- p2_rec*log(1.1) #### account for composite rec variab

  # for mixed models, the above functions will predict site-specific results (i.e., will incorpor
  rm(s_data_for_prediction) # this part removes the site-specific dataframe used in this loop, so
}

predicted_sur <- apply(predicted_sur_allsites, MARGIN= 2, FUN= mean, na.rm=T) # average size-spec
predicted_gr <- apply(predicted_gr_allsites, MARGIN= 2, FUN= mean, na.rm=T)
predicted_pr <- apply(predicted_pr_allsites, MARGIN= 2, FUN= mean, na.rm=T)
predicted_cr <- apply(predicted_cr_allsites, MARGIN= 2, FUN= mean, na.rm=T)
predicted_vg <- apply(predicted_vg_allsites, MARGIN= 2, FUN= mean, na.rm=T)
predicted_vg[which(predicted_vg<0)] <- 0 #### to prevent negative variance
predicted_rec <- mean(predicted_rec_allsites, na.rm=T)

gmx <- matrix(NA, length(bin_mids),length(bin_mids)) ## create empty gmx before entering the no_b

# make the TSF-specific matrix
for (ss in 1:no_bins) {

```

```

growcdf <- pnorm(bin_edges,predicted_gr[ss],sqrt(predicted_vg[ss])) # Yes. Vargrowth=(pred - ob
grows <- growcdf[2:length(bin_edges)]-growcdf[1:(length(bin_edges)-1)]
if(sum(grows)>0){grows <- grows/sum(grows)
gmxf[,ss] <- grows
} else if (sum(grows)==0 & predicted_gr[ss]< min(bin_edges)) {
  gmxf[,ss] <- c(1, rep(0, length(bin_mids)-1))
} else if (sum(grows)==0 & predicted_gr[ss]> max(bin_edges)) {
  gmxf[,ss] <- c(rep(0, length(bin_mids)-1), 1) } else { # deals with complete eviction-- OUT D
    gmxf[,ss] <- NA}
  # this if statement breaks the code (puts NA's into the matrix) if the sum of the PDF is zero (
} # end ss loop

# make the surv*growth mx
survgmx <- gmxf*t(matrix( rep(predicted_sur,no_bins),no_bins)) # Yes, this is doing element-wise m
repro <- predicted_sur*predicted_pr*predicted_cr # prob that you survive, reproduce, then number

mx <- matrix(0, no_bins+1, no_bins+1)
mx[2:(no_bins+1), 2:(no_bins+1)] = survgmx
mx[1,2:(no_bins+1)] = repro # FRUIT ROW

sdlng_cdf <- pnorm(bin_edges,mean_seedling_logsize,sqrt(var_seedling_logsize))
sdlng_pdf <- sdlng_cdf[2:length(bin_edges)]-sdlng_cdf[1:(length(bin_edges)-1)]
if(sum(sdlng_pdf)>0){
  sdlng_pdf <- sdlng_pdf/sum(sdlng_pdf)} else {
  sdlng_pdf <- NA}

mx[2:(no_bins+1),1] = predicted_rec *sdlng_pdf

mx_prod <- mx_prod %*% mx # in the first iteration of the loop, mx just becomes mx_prod

} # closes ii (TSFs) loop

j_pop_gr_rate_FB_reg[i] <- (popbio::lambda(mx_prod))^(1/length(TSFs))
# multiply them together to get a matrix, etc.
}
j_pop_gr_rate_FB_reg
}

```

2.3.2 FB IPM

```

all_pop_growth_rates_FB_reg <- cbind(possible_scenarios, all_pop_growth_rates_FB_reg)
names(all_pop_growth_rates_FB_reg) <- c(names(possible_scenarios), paste("rep", 1:no_reps, sep = ""))
rep_cols <- grep("^rep", names(all_pop_growth_rates_FB_reg), value = TRUE)

mean_lambda_across_scenarios_FB_reg <- all_pop_growth_rates_FB_reg %>%
  rowwise() %>%
  mutate(
    mean_lambda = mean(c_across(all_of(rep_cols)), na.rm = TRUE),
    lower_bound = quantile(c_across(all_of(rep_cols)), 0.025, na.rm = TRUE),
    upper_bound = quantile(c_across(all_of(rep_cols)), 0.975, na.rm = TRUE)
  ) %>%

```

```

dplyr::select(FRI,s.TBF,mean_lambda,lower_bound, upper_bound) %>%
mutate(TBF=NA) %>%
ungroup()

mean_lambda_across_scenarios_FB_reg$TBF[1:11] <- "Short TBF"
mean_lambda_across_scenarios_FB_reg$TBF[12:22] <- "Long TBF"

#####. graph
#####

plt7<- ggplot(data= mean_lambda_across_scenarios_FB_reg, aes(x= FRI, y= mean_lambda)) +
  geom_line(aes(color= TBF)) +
  geom_ribbon(aes(ymin = mean_lambda_across_scenarios_FB_reg$lower_bound, ymax = mean_lambda_across_scenarios_FB_reg$upper_bound, fill = TBF)) +
  #ylim(0,1) +
  #xlim(c(0, 12)) +
  labs(y= "Growth Rate", x = "Fire Return Interval",color = NULL, fill = NULL) +
  theme_bw() + theme(legend.position='none') +
  theme(legend.position = "right") +
  scale_x_continuous(breaks = 0:10) +
  theme(text = element_text(size = 16))
plt7

```

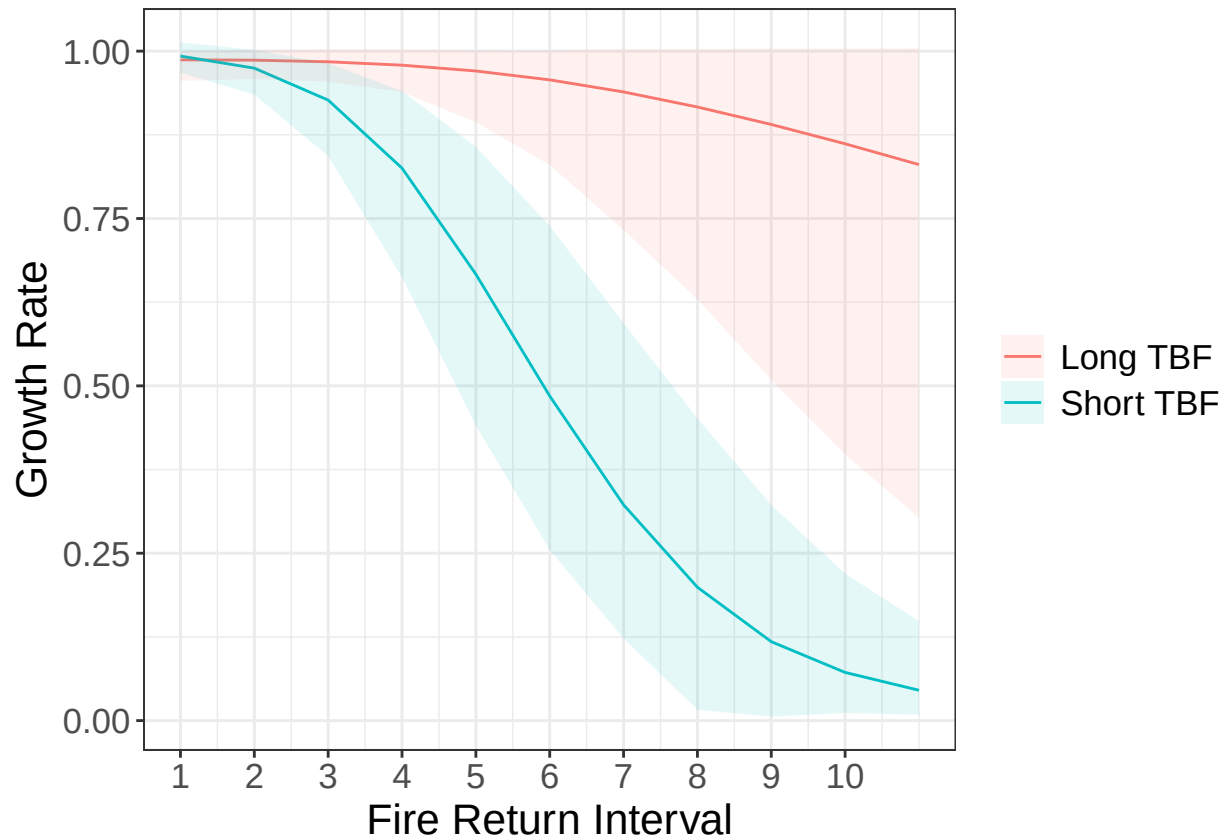
2.3.3 graph

```

## Warning: Use of 'mean_lambda_across_scenarios_FB_reg$lower_bound' is discouraged.
## i Use 'lower_bound' instead.

## Warning: Use of 'mean_lambda_across_scenarios_FB_reg$upper_bound' is discouraged.
## i Use 'upper_bound' instead.

```



3. Result visualization

Now we have created three region specific IPMs. For the final step, we want to compare predictions of sub-IPMs to see how good our global IPM is at capturing region level responses of plant population dynamics to fire.

```
lambda_regionalIPM <- bind_rows(
  mean_lambda_across_scenarios %>% mutate(dataset = "global IPM"),
  mean_lambda_across_scenarios_Cro_reg %>% mutate(dataset = "Croatan"),
  mean_lambda_across_scenarios_GSP_reg %>% mutate(dataset = "Green Swamp"),
  mean_lambda_across_scenarios_FB_reg %>% mutate(dataset = "Fort Bragg"))

plt8 <- ggplot(lambda_regionalIPM, aes(x = FRI, y = mean_lambda)) +
  geom_ribbon(
    aes(ymin = lower_bound, ymax = upper_bound,
        fill = dataset, group = interaction(dataset, TBF)),
    alpha = 0.1) +
  geom_line(
    aes(color = dataset, linetype = TBF,
        group = interaction(dataset, TBF)),
    size = 1) +
  theme_bw() +
  labs(
    y = "Growth Rate",
    x = "Fire Return Interval",
```

```

color = "Dataset",
fill = "Dataset",
linetype = "TBF" +
scale_x_continuous(breaks = 0:10) +
theme(text = element_text(size = 16))

```

plt8

